

Assignment 1 - PostgreSQL Basics

1. General Guidelines

1. Use snake_case naming convention for all database objects.
2. Primary keys must be UUID.
3. Use PostgreSQL SQL syntax.
4. Follow proper auditing practices.

2. Instruction Executed

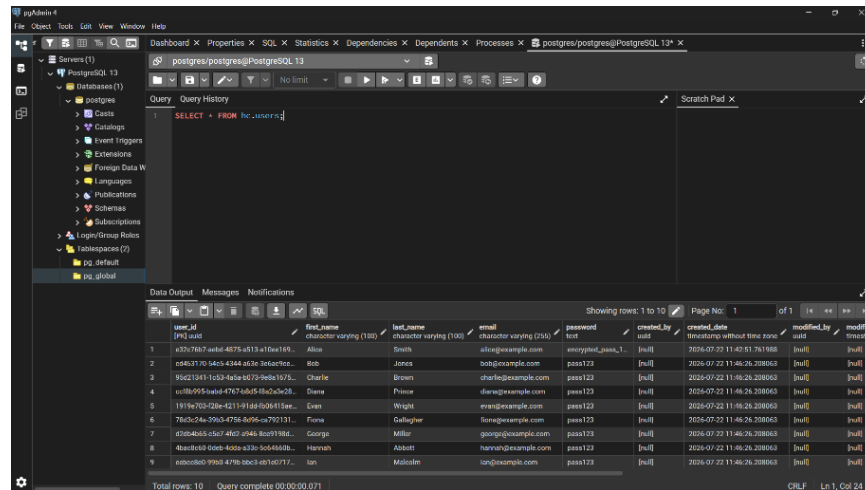
1. CREATE DATABASE homedcare_db;
2. CREATE SCHEMA hc;
3. CREATE TABLE hc.users (
user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
first_name VARCHAR(100),
last_name VARCHAR(100),
email VARCHAR(255) UNIQUE,
password TEXT,
created_by UUID,
created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
modified_by UUID,
modified_date TIMESTAMP,
is_active BOOLEAN DEFAULT TRUE,
is_deleted BOOLEAN DEFAULT FALSE,
birth_date DATE,
address TEXT,
mobile_number VARCHAR(20)
);
4. SELECT * FROM hc.users;
5. INSERT INTO hc.users (
first_name,
last_name,
email,
password,
birth_date,
address,
mobile_number
) VALUES (
'Alice',
'Smith',
'alice@example.com',
'encrypted_pass_123',
'1985-06-15',
'123 Maple Street',
'555-1234'
);

Work done by: Mitesh Bhagat

6. INSERT INTO hc.users (first_name, last_name, email, password, birth_date, address, mobile_number)

VALUES

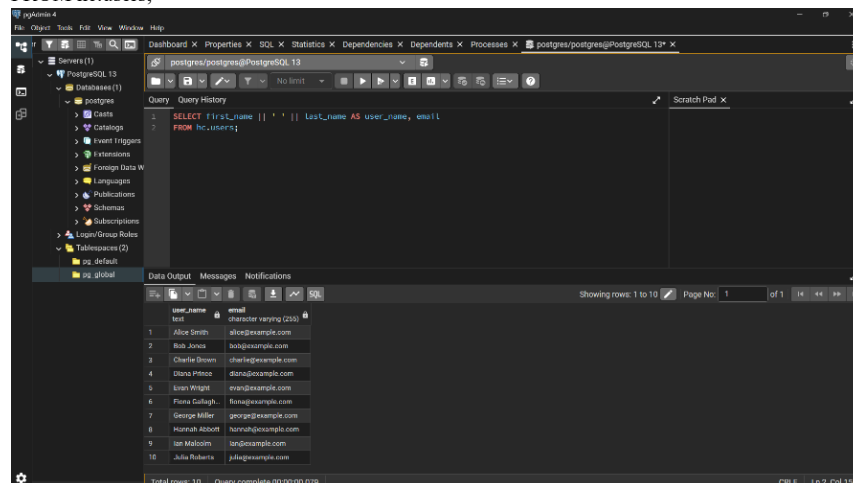
('Bob', 'Jones', 'bob@example.com', 'pass123', '1990-08-20', '456 Oak Ave', '555-5678'),
('Charlie', 'Brown', 'charlie@example.com', 'pass123', '1975-12-05', '789 Pine Rd', '555-9012'),
('Diana', 'Prince', 'diana@example.com', 'pass123', '1988-03-22', '321 Elm St', '555-3456'),
('Evan', 'Wright', 'evan@example.com', 'pass123', '1995-11-10', '654 Birch Blvd', '555-7890'),
('Fiona', 'Gallagher', 'fiona@example.com', 'pass123', '1992-07-08', '987 Cedar Ln', '555-2345'),
('George', 'Miller', 'george@example.com', 'pass123', '1980-02-14', '147 Spruce Ct', '555-6789'),
('Hannah', 'Abbott', 'hannah@example.com', 'pass123', '1998-09-30', '258 Willow Dr', '555-0123'),
('Ian', 'Malcolm', 'ian@example.com', 'pass123', '1970-05-18', '369 Aspen Way', '555-4567'),
('Julia', 'Roberts', 'julia@example.com', 'pass123', '1982-10-25', '741 Cherry Pl', '555-8901');



The screenshot shows the pgAdmin 4 interface with the 'Query History' tab selected. The query executed was `SELECT * FROM hc.users;`. The results are displayed in a table with 10 rows and 10 columns. The columns are: user_id, first_name, last_name, email, password, birth_date, address, mobile_number, created_by, and created_date. The data is as follows:

user_id	first_name	last_name	email	password	birth_date	address	mobile_number	created_by	created_date
1	Bob	Jones	bob@example.com	pass123	1990-08-20	456 Oak Ave	555-5678	[null]	2024-07-22 11:40:51.761888
2	Charlie	Brown	charlie@example.com	pass123	1975-12-05	789 Pine Rd	555-9012	[null]	2024-07-22 11:46:26.208063
3	Diana	Prince	diana@example.com	pass123	1988-03-22	321 Elm St	555-3456	[null]	2024-07-22 11:46:26.208063
4	Evan	Wright	evan@example.com	pass123	1995-11-10	654 Birch Blvd	555-7890	[null]	2024-07-22 11:46:26.208063
5	Fiona	Gallagher	fiona@example.com	pass123	1992-07-08	987 Cedar Ln	555-2345	[null]	2024-07-22 11:46:26.208063
6	George	Miller	george@example.com	pass123	1980-02-14	147 Spruce Ct	555-6789	[null]	2024-07-22 11:46:26.208063
7	Hannah	Abbott	hannah@example.com	pass123	1998-09-30	258 Willow Dr	555-0123	[null]	2024-07-22 11:46:26.208063
8	Ian	Malcolm	ian@example.com	pass123	1970-05-18	369 Aspen Way	555-4567	[null]	2024-07-22 11:46:26.208063
9	Julia	Roberts	julia@example.com	pass123	1982-10-25	741 Cherry Pl	555-8901	[null]	2024-07-22 11:46:26.208063
10									

7. SELECT first_name || ' ' || last_name AS user_name, email
FROM hc.users;

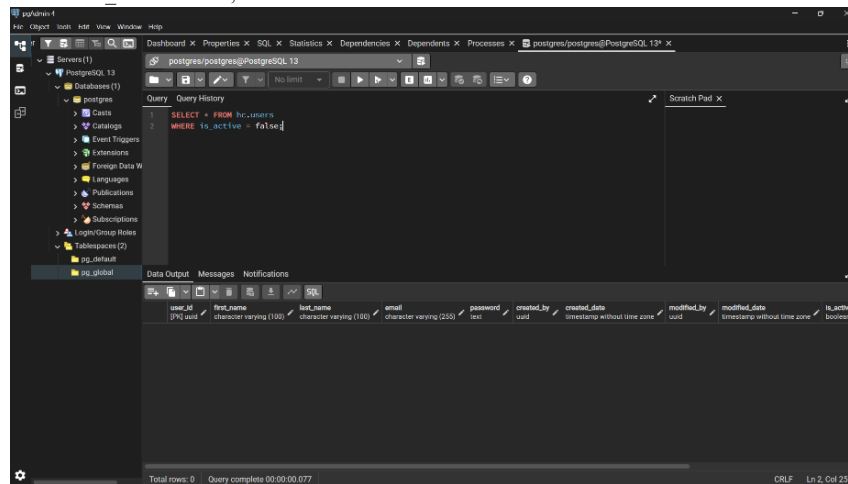


The screenshot shows the pgAdmin 4 interface with the 'Query History' tab selected. The query executed was `SELECT first_name || ' ' || last_name AS user_name, email FROM hc.users;`. The results are displayed in a table with 10 rows and 2 columns. The columns are: user_name and email. The data is as follows:

user_name	email
Bob Jones	bob@example.com
Charlie Brown	charlie@example.com
Diana Prince	diana@example.com
Evan Wright	evan@example.com
Fiona Gallagher	fiona@example.com
George Miller	george@example.com
Hannah Abbott	hannah@example.com
Ian Malcolm	ian@example.com
Julia Roberts	julia@example.com

Work done by: Mitesh Bhagat

8. `SELECT * FROM hc.users`
`WHERE is_active = false;`

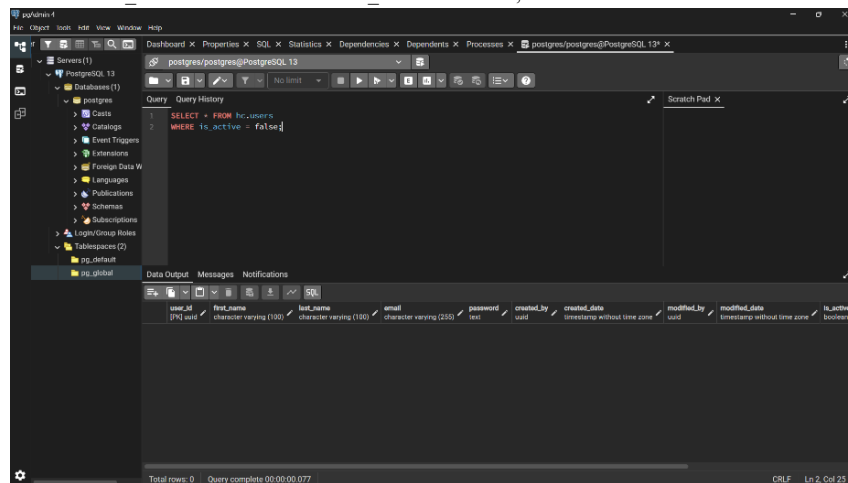


Query: `SELECT * FROM hc.users WHERE is_active = false;`

user_id	first_name	last_name	email	password	created_by	created_date	modified_by	modified_date	is_active
---------	------------	-----------	-------	----------	------------	--------------	-------------	---------------	-----------

Total rows: 0 Query complete 00:00:00.077

9. `SELECT * FROM hc.users`
`WHERE first_name LIKE 'A%' AND last_name LIKE '%i';`

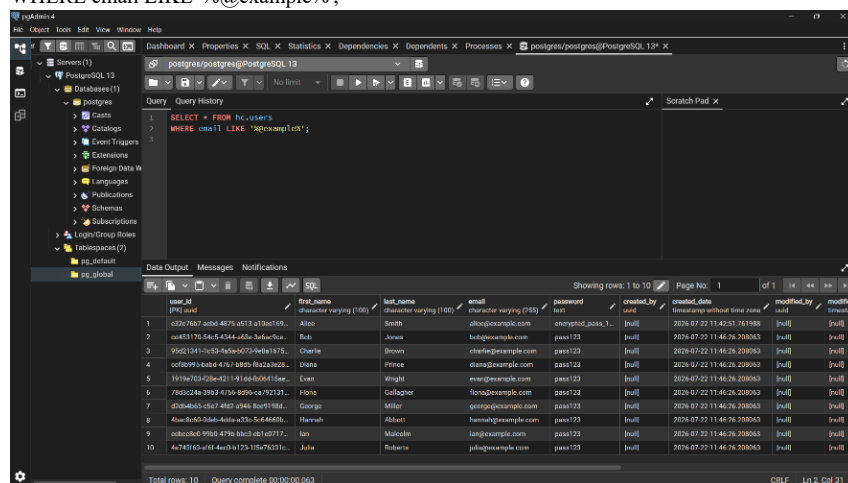


Query: `SELECT * FROM hc.users WHERE first_name LIKE 'A%' AND last_name LIKE '%i';`

user_id	first_name	last_name	email	password	created_by	created_date	modified_by	modified_date	is_active
---------	------------	-----------	-------	----------	------------	--------------	-------------	---------------	-----------

Total rows: 0 Query complete 00:00:00.077

10. `SELECT * FROM hc.users`
`WHERE email LIKE '%@example%';`



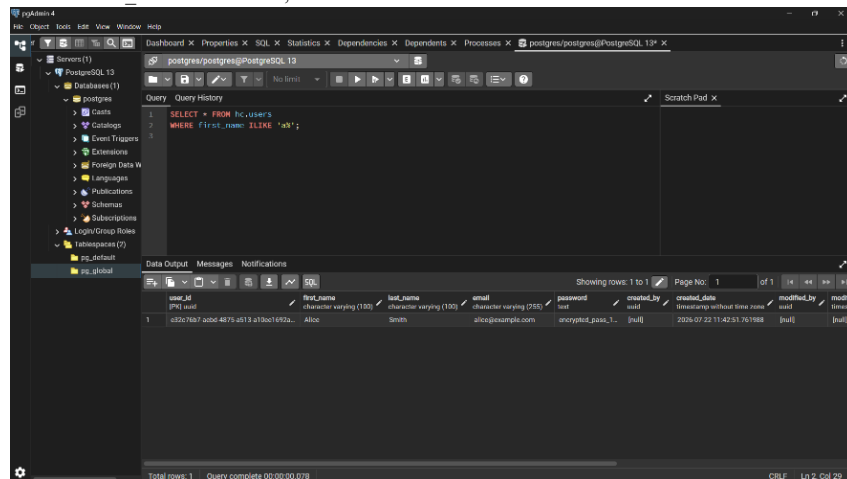
Query: `SELECT * FROM hc.users WHERE email LIKE '%@example%';`

user_id	first_name	last_name	email	password	created_by	created_date	modified_by	modified_date	is_active
1	Aliso	Smith	aliso@example.com	encrypted_pass_1	[null]	2026-07-22 11:42:51.761188	[null]	[null]	[null]
2	Bob	Jones	bob@example.com	pass123	[null]	2026-07-22 11:46:36.208063	[null]	[null]	[null]
3	Charlie	Brown	charlie@example.com	pass123	[null]	2026-07-22 11:46:26.208063	[null]	[null]	[null]
4	Diana	Prince	diana@example.com	pass123	[null]	2026-07-22 11:46:26.208063	[null]	[null]	[null]
5	Eva	King	eva@example.com	pass123	[null]	2026-07-22 11:46:26.208063	[null]	[null]	[null]
6	Fiona	Callaghan	fiona@example.com	pass123	[null]	2026-07-22 11:46:26.208063	[null]	[null]	[null]
7	George	Miller	george@example.com	pass123	[null]	2026-07-22 11:46:26.208063	[null]	[null]	[null]
8	Hannah	Abbot	hannah@example.com	pass123	[null]	2026-07-22 11:46:26.208063	[null]	[null]	[null]
9	Ian	Malcolm	ian@example.com	pass123	[null]	2026-07-22 11:46:26.208063	[null]	[null]	[null]
10	Julia	Rubiera	julia@example.com	pass123	[null]	2026-07-22 11:46:26.208063	[null]	[null]	[null]

Total rows: 10 Query complete 00:00:00.063

Work done by: Mitesh Bhagat

11. SELECT * FROM hc.users
WHERE first_name ILIKE 'a%';



Query:

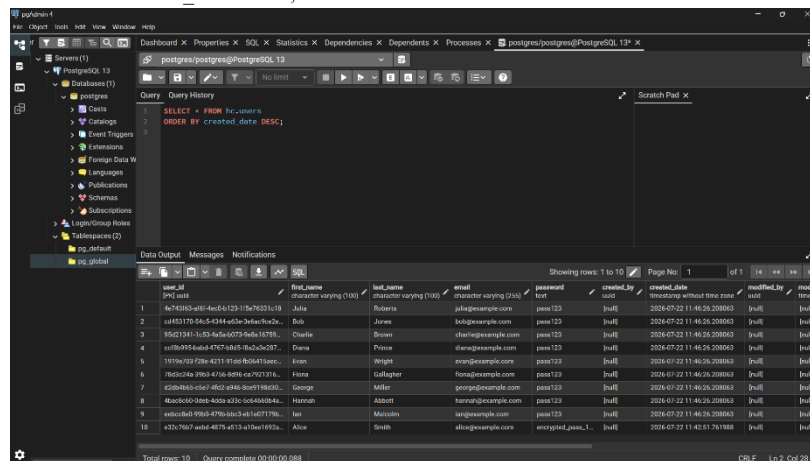
```
1 SELECT * FROM hc.users
2 WHERE first_name ILIKE 'a%';
```

Data Output:

user_id	first_name	last_name	email	password	created_by	created_date	modified_by	modified_date
1	Alice	Smith	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.761988	[null]	[null]

Total rows: 1 Query complete 00:00:00.079

12. SELECT * FROM hc.users
ORDER BY created_date DESC;



Query:

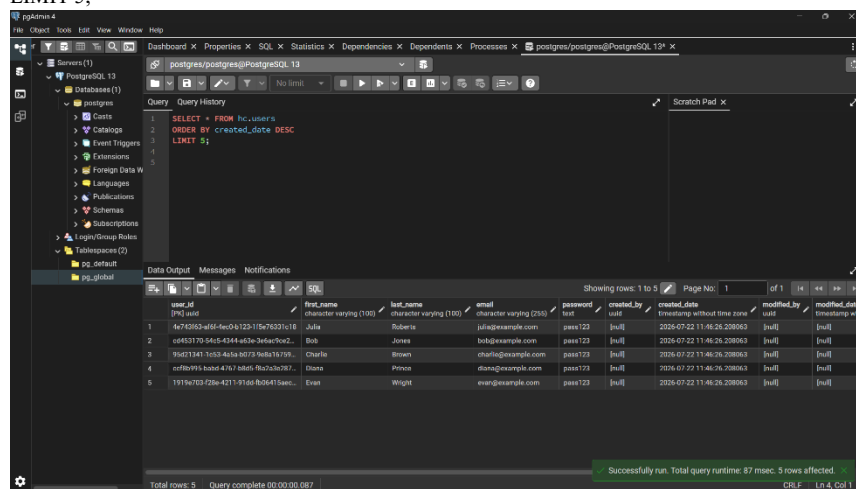
```
1 SELECT * FROM hc.users
2 ORDER BY created_date DESC;
```

Data Output:

user_id	first_name	last_name	email	password	created_by	created_date	modified_by	modified_date
1	Alice	Smith	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.761988	[null]	[null]
2	Bob	Jones	bob@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
3	Charlie	Brown	charlie@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
4	Diana	Prince	diana@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
5	Evan	Wright	evan@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
6	Fiona	Gallagher	fiona@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
7	George	Miller	george@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
8	Harmon	Abdott	harmon@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
9	Ian	Malcolm	ian@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
10	Alice	Smith	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.761988	[null]	[null]

Total rows: 10 Query complete 00:00:00.088

13. SELECT * FROM hc.users
ORDER BY created_date DESC
LIMIT 5;



Query:

```
1 SELECT * FROM hc.users
2 ORDER BY created_date DESC
3 LIMIT 5;
```

Data Output:

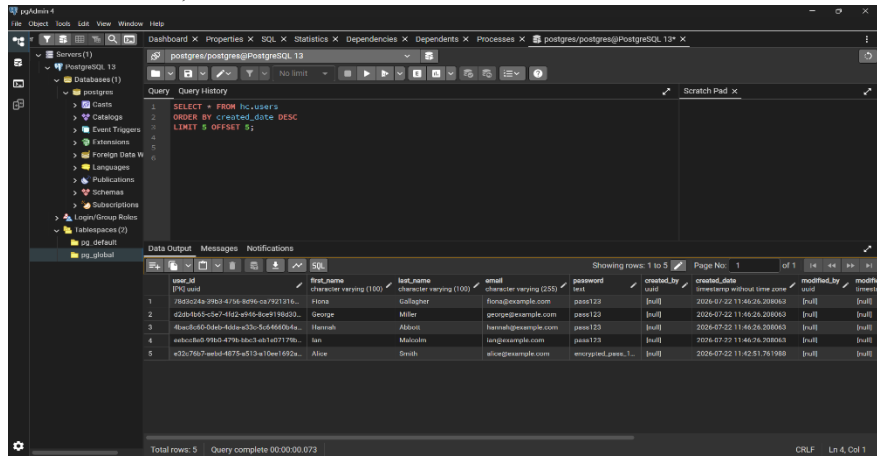
user_id	first_name	last_name	email	password	created_by	created_date	modified_by	modified_date
1	Alice	Smith	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.761988	[null]	[null]
2	Bob	Jones	bob@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
3	Charlie	Brown	charlie@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
4	Diana	Prince	diana@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
5	Evan	Wright	evan@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]

Total rows: 5 Query complete 00:00:00.087

Successfully run. Total query runtime: 87 msec. 5 rows affected.

Work done by: Mitesh Bhagat

14. SELECT * FROM hc.users
ORDER BY created_date DESC
LIMIT 5 OFFSET 5;



The screenshot shows the pgAdmin 4 interface with a query executed in the 'Query' tab. The query is: `SELECT * FROM hc.users ORDER BY created_date DESC LIMIT 5 OFFSET 5;`. The results are displayed in the 'Data Output' tab, showing 5 rows of user data. The table has columns: user_id, first_name, last_name, email, password, created_by, created_date, and modified_by. The data is sorted by created_date in descending order, starting from the 6th row (offset 5).

user_id	first_name	last_name	email	password	created_by	created_date	modified_by
1	John	Doe	john.doe@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
2	George	Miller	george@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
3	Hannah	Abbot	hannah@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
4	Jan	Matson	jan@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
5	Alice	Bradt	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.781980	[null]

Assignment 2 - PostgreSQL Basics

General Instructions

- Use PostgreSQL syntax only
- Follow snake_case naming convention
- Do not include query outputs
- Focus on correct SQL logic
- Database: training_db
- Schema: tdb

Available Tables

- users
 - roles
 - service_types
 - categories
 - sub_categories
- Instruction Executed

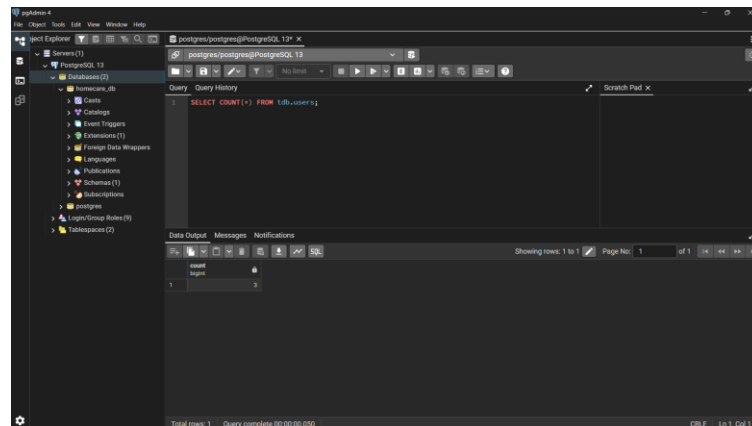
3. Instruction Executed

1. Importing form Folder

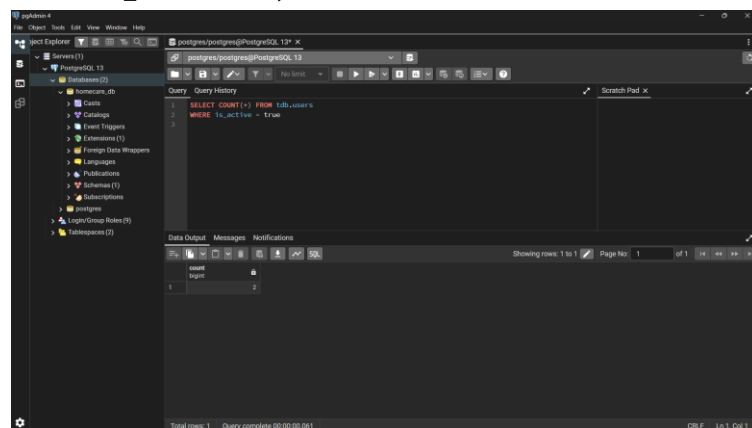
1. File Name: "PostgreSQL Assignment 1"

2. Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

1. SELECT COUNT(*) FROM tdb.users;

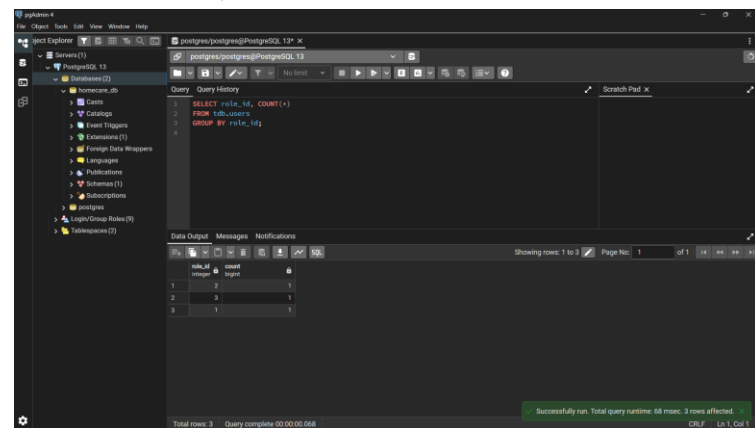


2. SELECT COUNT(*) FROM tdb.users
WHERE is_active = true;



Work done by: Mitesh Bhagat

3. `SELECT role_id, COUNT(*)`
`FROM tdb.users`
`GROUP BY role_id;`



The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL query:

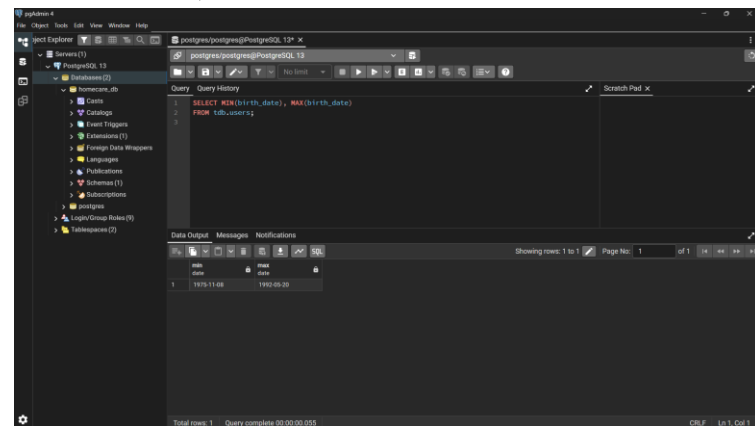
```
1 SELECT role_id, COUNT(*)
2 FROM tdb.users;
3 GROUP BY role_id;
```

The Data Output pane shows the results of the query:

role_id	count
1	3
2	1
3	1

At the bottom, a status bar indicates: "Successfully ran. Total query runtime: 68 msec. 3 rows affected. CRLF Ln 1, Col 1".

4. `SELECT MIN(birth_date), MAX(birth_date)`
`FROM tdb.users;`



The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL query:

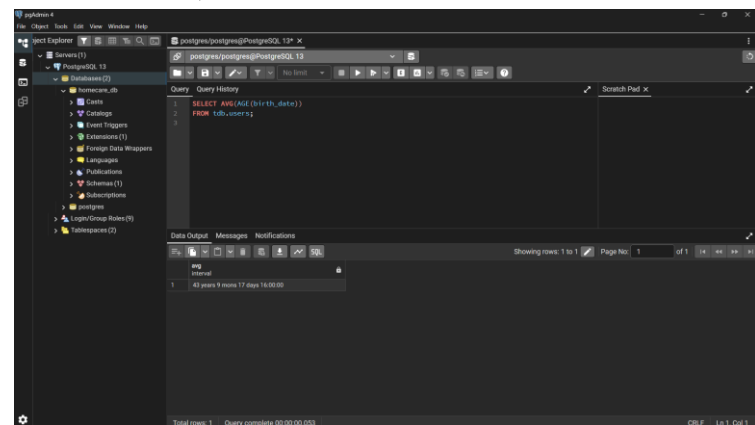
```
1 SELECT MIN(birth_date), MAX(birth_date)
2 FROM tdb.users;
```

The Data Output pane shows the results of the query:

min_date	max_date
1976-11-08	1992-05-20

At the bottom, a status bar indicates: "Total rows: 1 Query complete 00:00:00.055 CRLF Ln 1, Col 1".

5. `SELECT AVG(AGE(birth_date))`
`FROM tdb.users;`



The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL query:

```
1 SELECT AVG(AGE(birth_date))
2 FROM tdb.users;
```

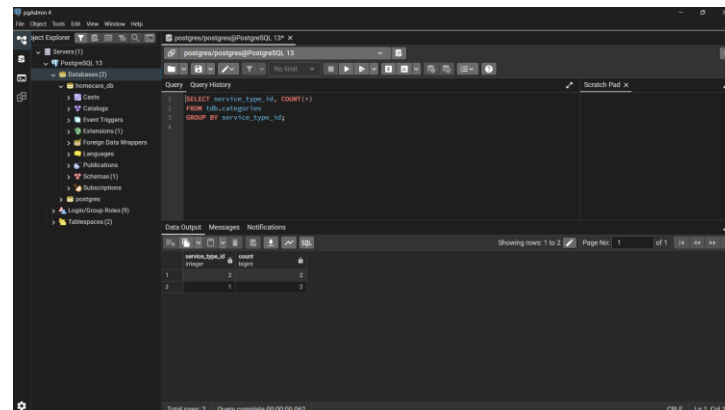
The Data Output pane shows the results of the query:

avg_age
42 years 9 months 17 days 16:00:00

At the bottom, a status bar indicates: "Total rows: 1 Query complete 00:00:00.053 CRLF Ln 1, Col 1".

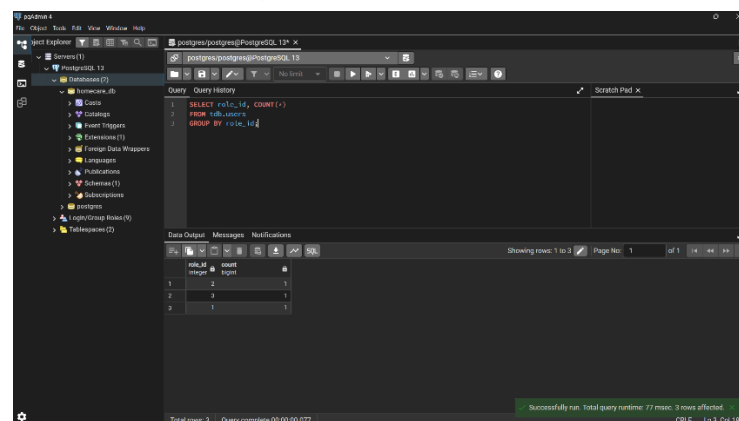
Work done by: Mitesh Bhagat

6. `SELECT service_type_id, COUNT(*)`
`FROM tdb.categories`
`GROUP BY service_type_id;`

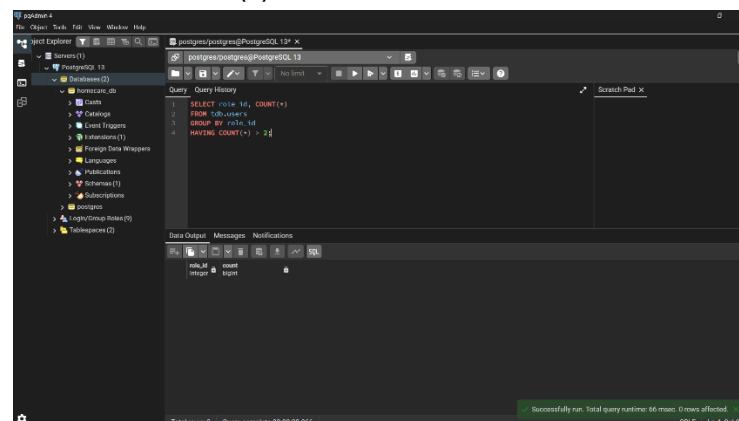


3. GROUP BY & HAVING

1. `SELECT role_id, COUNT(*)`
`FROM tdb.users`
`GROUP BY role_id;`



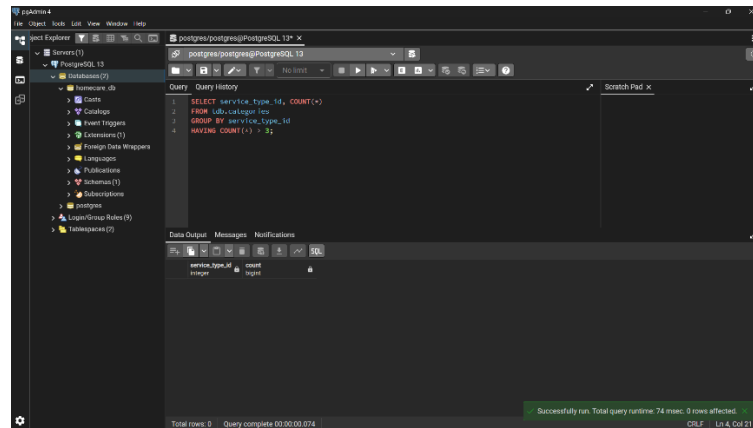
2. `SELECT role_id, COUNT(*)`
`FROM tdb.users`
`GROUP BY role_id`
`HAVING COUNT(*) > 2;`



Work done by: Mitesh Bhagat

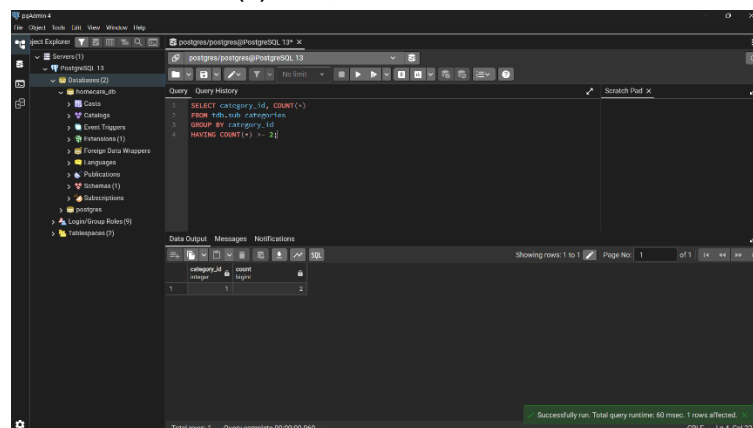
3.

```
SELECT service_type_id, COUNT(*)  
FROM tdb.categories  
GROUP BY service_type_id  
HAVING COUNT(*) > 3;
```



4.

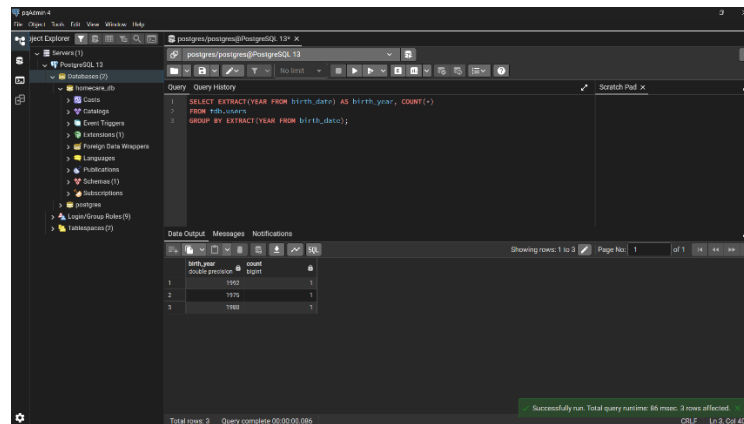
```
SELECT category_id, COUNT(*)  
FROM tdb.sub_categories  
GROUP BY category_id  
HAVING COUNT(*) >= 2;
```



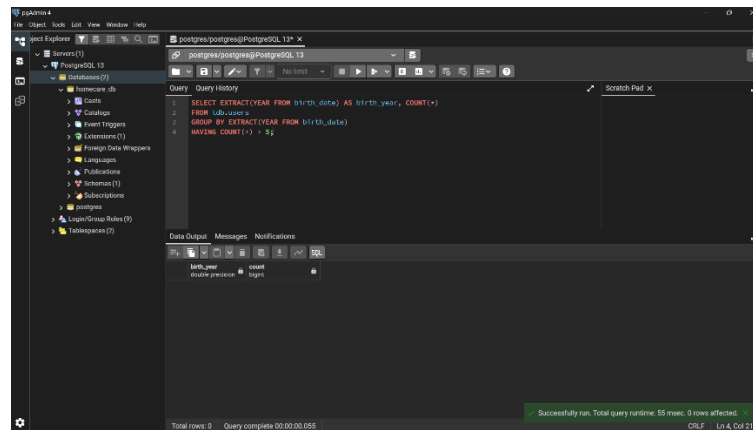
5.

```
SELECT EXTRACT(YEAR FROM birth_date) AS birth_year, COUNT(*)  
FROM tdb.users  
GROUP BY EXTRACT(YEAR FROM birth_date);
```

Work done by: Mitesh Bhagat



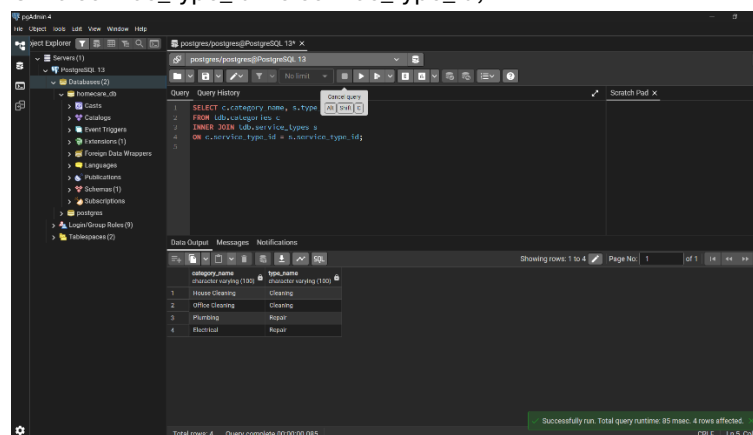
6. `SELECT EXTRACT(YEAR FROM birth_date) AS birth_year, COUNT(*)`
`FROM tdb.users`
`GROUP BY EXTRACT(YEAR FROM birth_date)`
`HAVING COUNT(*) > 5;`



4. Joins

1. **Inner Join: Fetch categories with service type names**

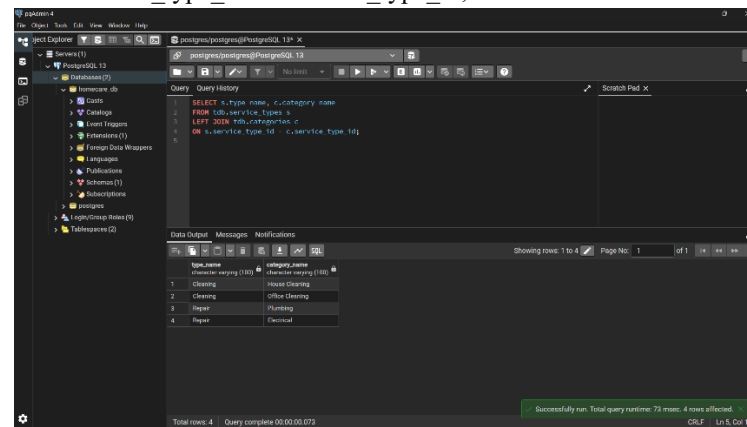
`SELECT c.category_name, s.type_name`
`FROM tdb.categories c`
`INNER JOIN tdb.service_types s`
`ON c.service_type_id = s.service_type_id;`



Work done by: Mitesh Bhagat

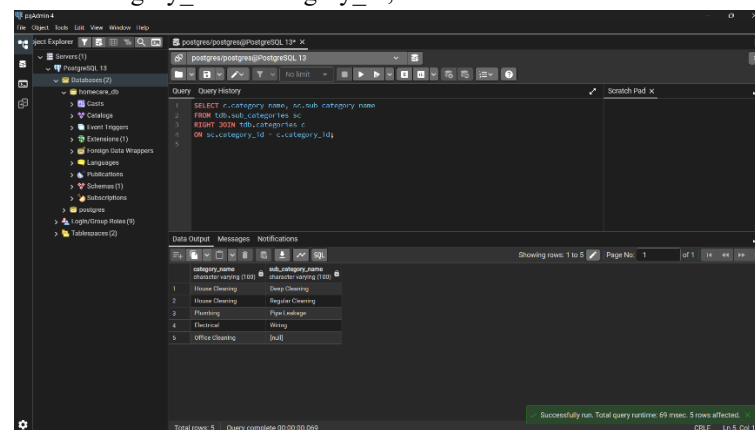
2. **Left Join: Fetch all service types including those without categories**

```
SELECT s.type_name, c.category_name
FROM tdb.service_types s
LEFT JOIN tdb.categories c
ON s.service_type_id = c.service_type_id;
```



3. **Right Join: Fetch all categories even if they have no sub-categories**

```
SELECT c.category_name, sc.sub_category_name
FROM tdb.sub_categories sc
RIGHT JOIN tdb.categories c
ON sc.category_id = c.category_id;
```

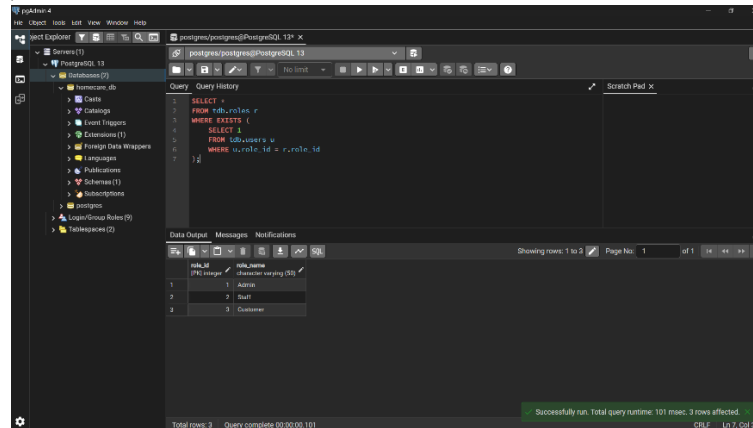


Work done by: Mitesh Bhagat

5. Exists

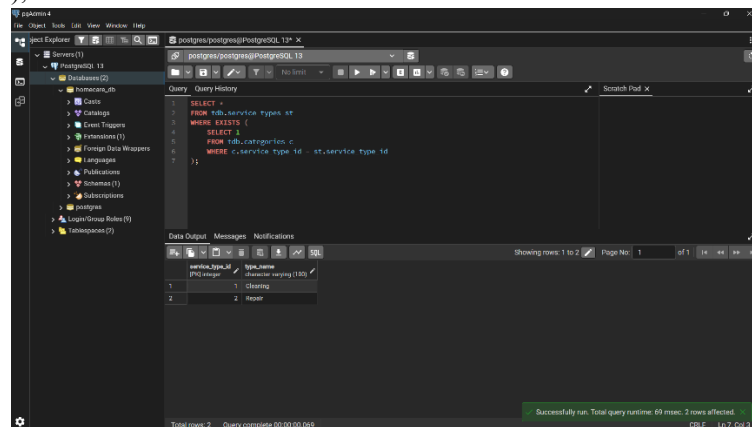
1.

```
SELECT *  
FROM tdb.roles r  
WHERE EXISTS (  
    SELECT 1  
    FROM tdb.users u  
    WHERE u.role_id = r.role_id  
);
```



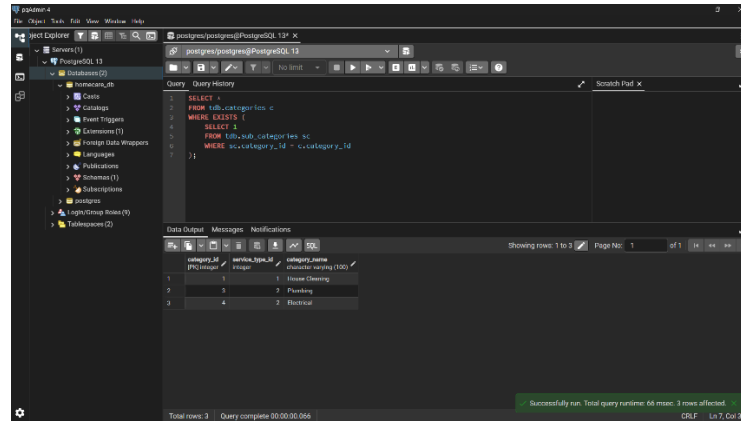
2.

```
SELECT *  
FROM tdb.service_types st  
WHERE EXISTS (  
    SELECT 1  
    FROM tdb.categories c  
    WHERE c.service_type_id = st.service_type_id  
);
```



Work done by: Mitesh Bhagat

3. SELECT *
FROM tdb.categories c
WHERE EXISTS (
SELECT 1
FROM tdb.sub_categories sc
WHERE sc.category_id = c.category_id
);



The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL query:

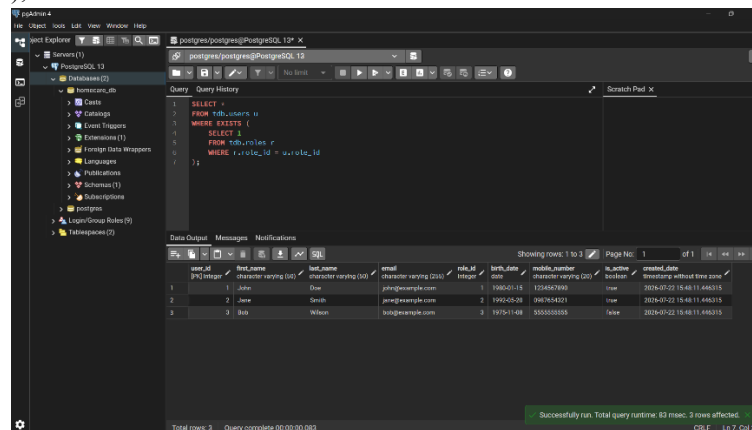
```
SELECT *  
FROM tdb.categories c  
WHERE EXISTS (  
SELECT 1  
FROM tdb.sub_categories sc  
WHERE sc.category_id = c.category_id  
);
```

The Data Output pane shows the results of the query:

category_id	category_name
1	House Cleaning
2	Painting
4	Electrical

Successfully run. Total query runtime: 66 msec. 3 rows affected.

4. SELECT *
FROM tdb.users u
WHERE EXISTS (
SELECT 1
FROM tdb.roles r
WHERE r.role_id = u.role_id
);



The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL query:

```
SELECT *  
FROM tdb.users u  
WHERE EXISTS (  
SELECT 1  
FROM tdb.roles r  
WHERE r.role_id = u.role_id  
);
```

The Data Output pane shows the results of the query:

user_id	first_name	last_name	email	role_id	birth_date	mobile_number	is_active	created_date
1	John	Doe	john@example.com	1	1980-01-15	1234567890	true	2024-01-22 15:48:11.446315
2	Jane	Smith	jane@example.com	2	1992-05-20	9876543210	true	2024-01-22 15:48:11.446315
3	Bob	Wilson	bob@example.com	2	1975-11-08	5555555555	false	2024-01-22 15:48:11.446315

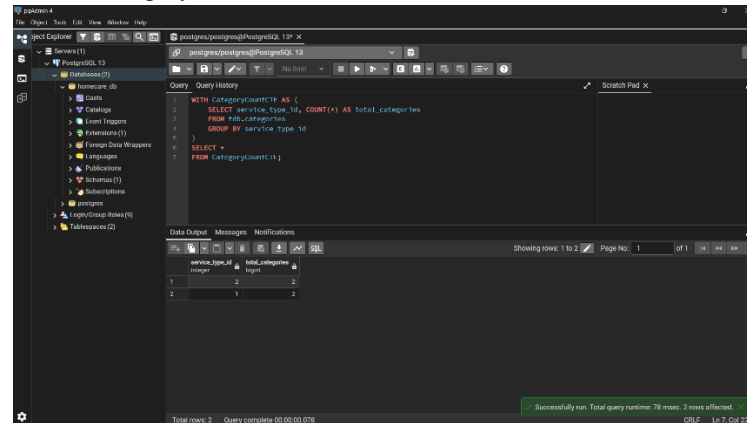
Successfully run. Total query runtime: 82 msec. 3 rows affected.

Work done by: Mitesh Bhagat

6. Common Table Expression

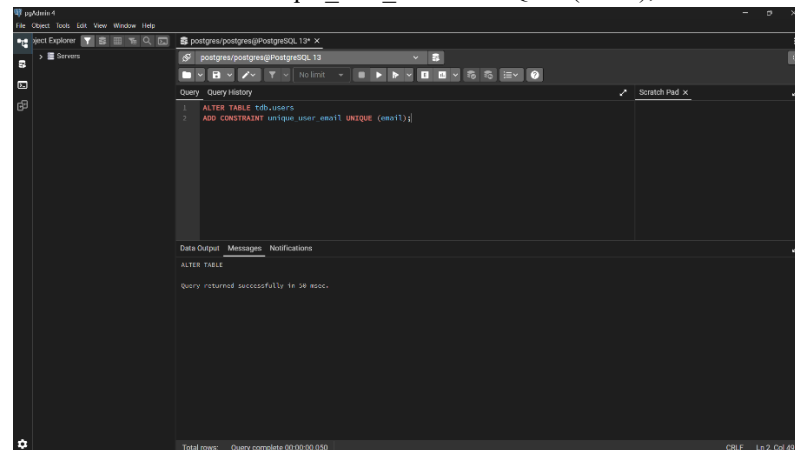
1. Use a CTE to count the number of categories per service type

```
WITH CategoryCountCTE AS (  
    SELECT service_type_id, COUNT(*) AS total_categories  
    FROM tdb.categories  
    GROUP BY service_type_id  
)  
SELECT *  
FROM CategoryCountCTE;
```



7. Constraints

1. ALTER TABLE tdb.users
ADD CONSTRAINT unique_user_email UNIQUE (email);



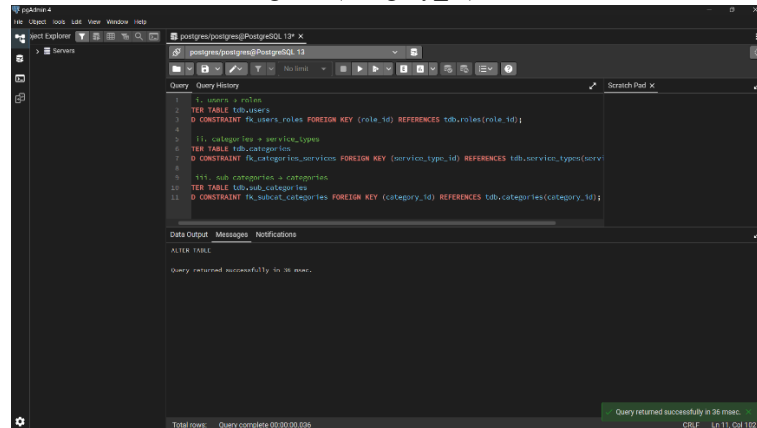
2. -- i. users → roles
ALTER TABLE tdb.users
ADD CONSTRAINT fk_users_roles FOREIGN KEY (role_id) REFERENCES
tdb.roles(role_id);

-- ii. categories → service_types
ALTER TABLE tdb.categories
ADD CONSTRAINT fk_categories_services FOREIGN KEY (service_type_id)
REFERENCES tdb.service_types(service_type_id);

-- iii. sub_categories → categories

Work done by: Mitesh Bhagat

```
ALTER TABLE tdb.sub_categories
ADD CONSTRAINT fk_subcat_categories FOREIGN KEY (category_id)
REFERENCES tdb.categories(category_id);
```

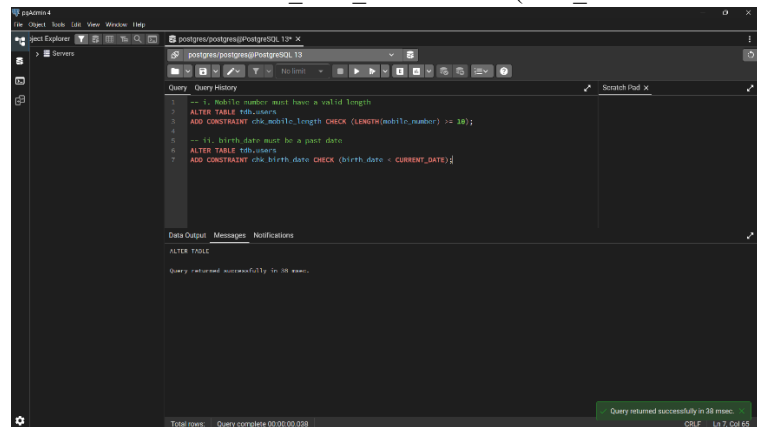


3. -- i. Mobile number must have a valid length

```
ALTER TABLE tdb.users
ADD CONSTRAINT chk_mobile_length CHECK (LENGTH(mobile_number) >=
10);
```

-- ii. birth_date must be a past date

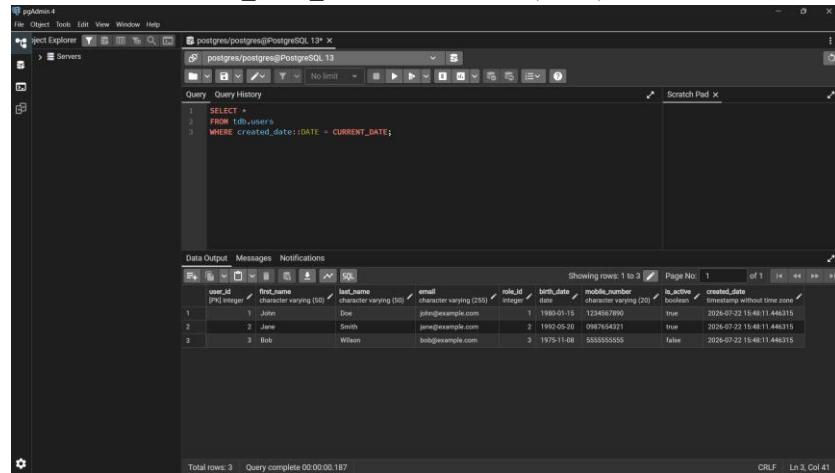
```
ALTER TABLE tdb.users
ADD CONSTRAINT chk_birth date CHECK (birth_date < CURRENT_DATE);
```



Work done by: Mitesh Bhagat

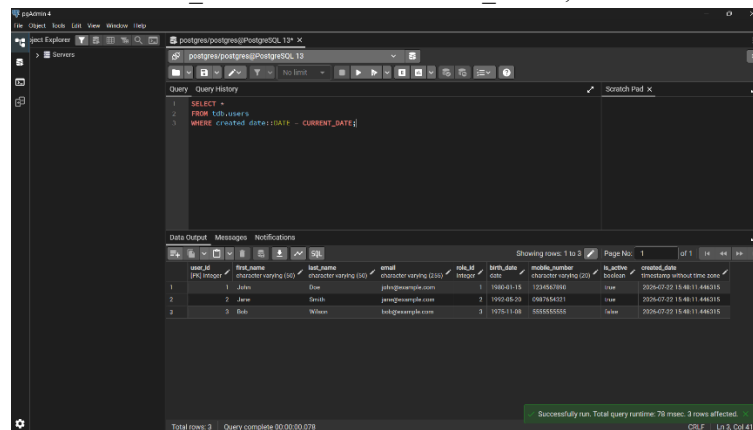
8. Indexes (Create an index on users.email)

1. CREATE INDEX idx_users_email ON tdb.users(email)

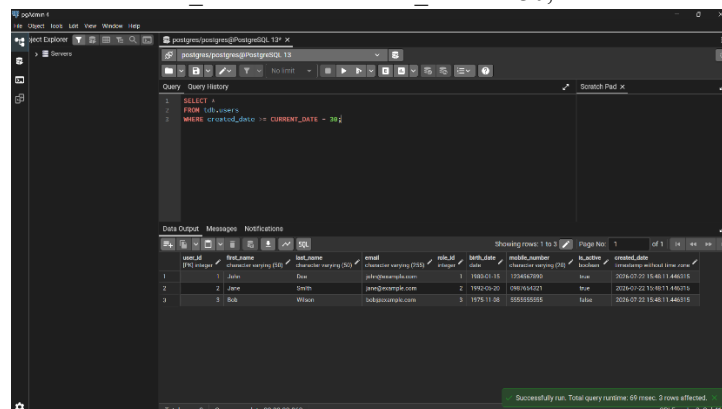


9. Date & Time Handling

1. SELECT * FROM tdb.users WHERE created_date::DATE = CURRENT_DATE;

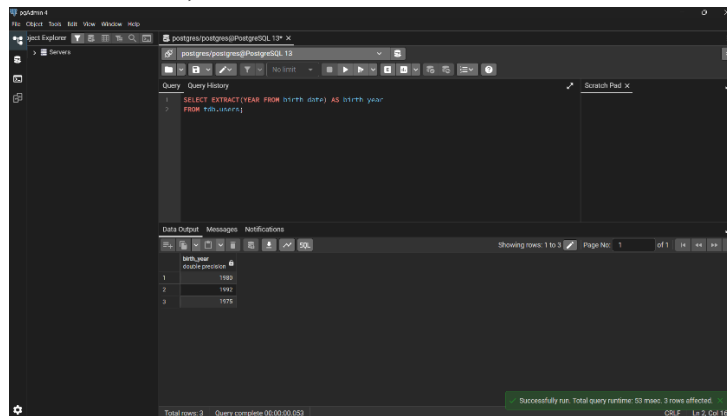


2. SELECT * FROM tdb.users WHERE created_date >= CURRENT_DATE - 30;

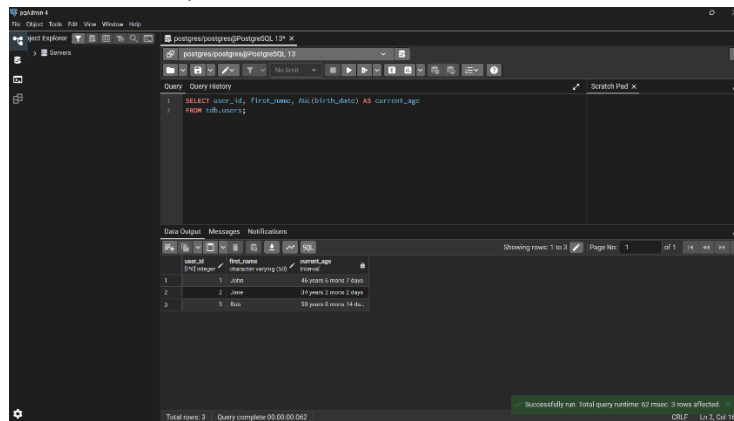


Work done by: Mitesh Bhagat

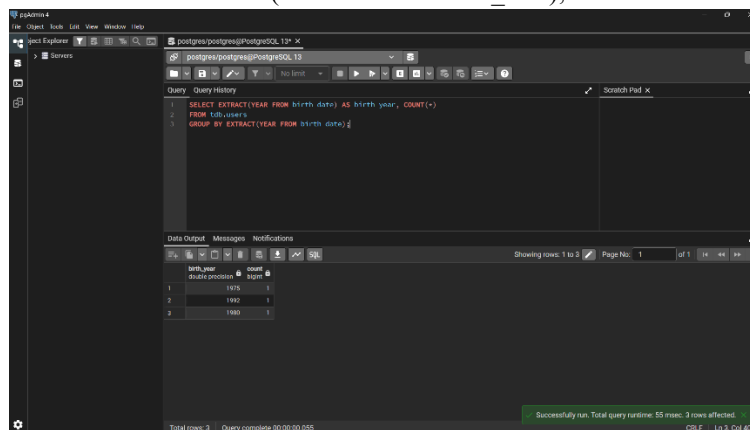
3. `SELECT EXTRACT(YEAR FROM birth_date) AS birth_year
FROM tdb.users;`



4. `SELECT user_id, first_name, AGE(birth_date) AS current_age
FROM tdb.users;`



5. `SELECT EXTRACT(YEAR FROM birth_date) AS birth_year, COUNT(*)
FROM tdb.users
GROUP BY EXTRACT(YEAR FROM birth_date);`



Work done by: Mitesh Bhagat

10. **Window Functions (ROW_NUMBER, RANK, LAG, LEAD)**

-- a. Assign row numbers to users within each role based on created_date

```
SELECT
    user_id,
    first_name,
    role_id,
    created_date,
    ROW_NUMBER() OVER(PARTITION BY role_id ORDER BY created_date) AS
row_num
FROM tdb.users;
```

-- b. Rank roles based on number of users

```
WITH RoleCounts AS (
    SELECT role_id, COUNT(*) AS total_users
    FROM tdb.users
    GROUP BY role_id
)
SELECT
    role_id,
    total_users,
    RANK() OVER(ORDER BY total_users DESC) AS role_rank
FROM RoleCounts;
```

-- c. Use LAG to fetch previous user creation date per role

```
SELECT
    user_id,
    role_id,
    created_date,
    LAG(created_date) OVER(PARTITION BY role_id ORDER BY created_date) AS
prev_created_date
FROM tdb.users;
```

-- d. Use LEAD to fetch next user creation date per role

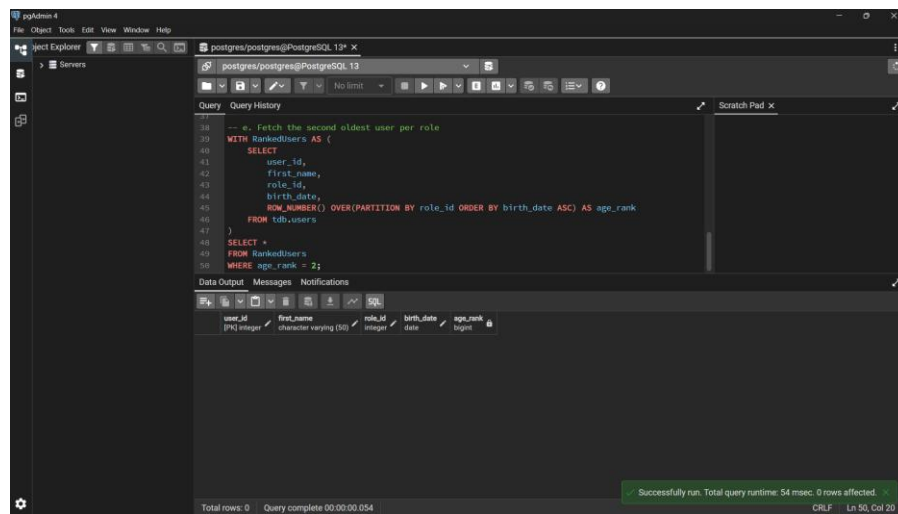
```
SELECT
    user_id,
    role_id,
    created_date,
    LEAD(created_date) OVER(PARTITION BY role_id ORDER BY created_date) AS
next_created_date
FROM tdb.users;
```

-- e. Fetch the second oldest user per role

```
WITH RankedUsers AS (
    SELECT
        user_id,
        first_name,
        role_id,
        birth_date,
        ROW_NUMBER() OVER(PARTITION BY role_id ORDER BY birth_date ASC) AS
age_rank
    FROM tdb.users
)
```

Work done by: Mitesh Bhagat

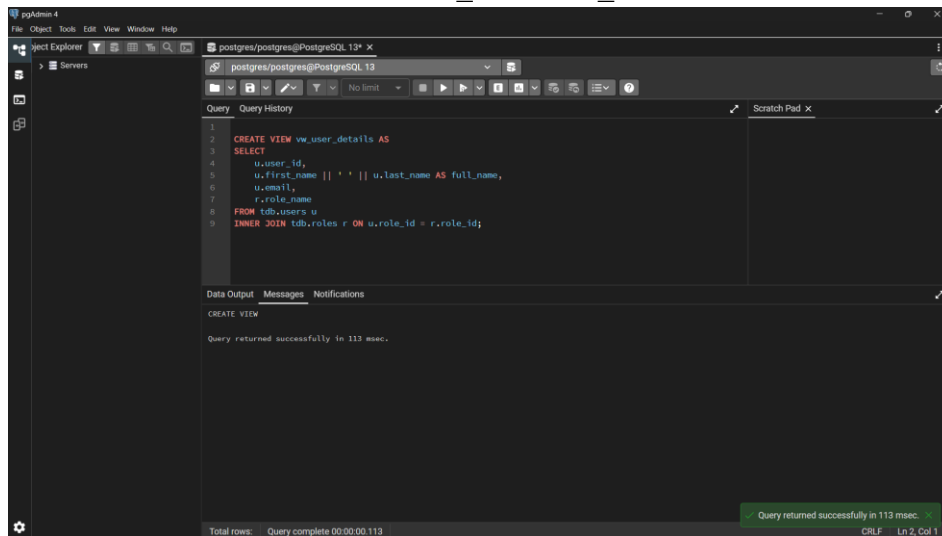
```
SELECT *  
FROM RankedUsers  
WHERE age_rank = 2;
```



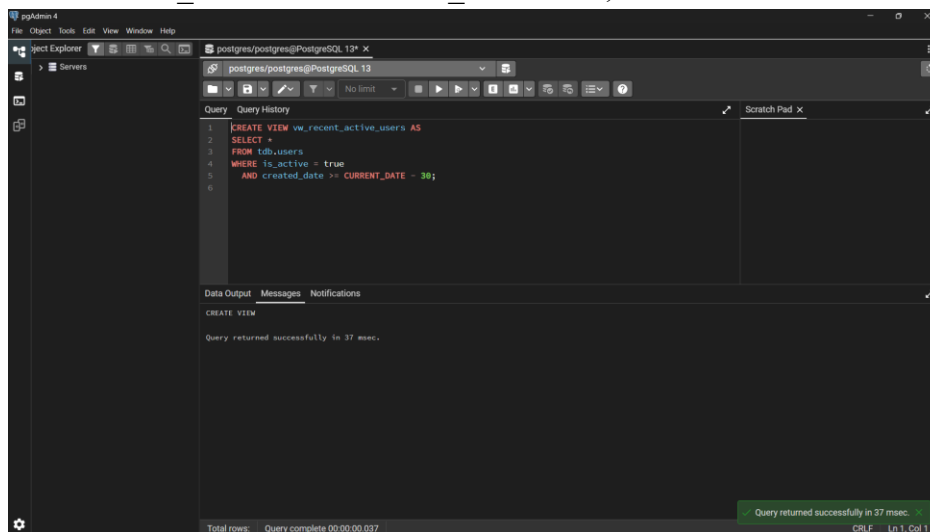
Assignment 3 - PostgreSQL Basics

1. Views & Materialized Views

1. CREATE VIEW vw_user_details AS
SELECT
 u.user_id,
 u.first_name || ' ' || u.last_name AS full_name,
 u.email,
 r.role_name
FROM tdb.users u
INNER JOIN tdb.roles r ON u.role_id = r.role_id;

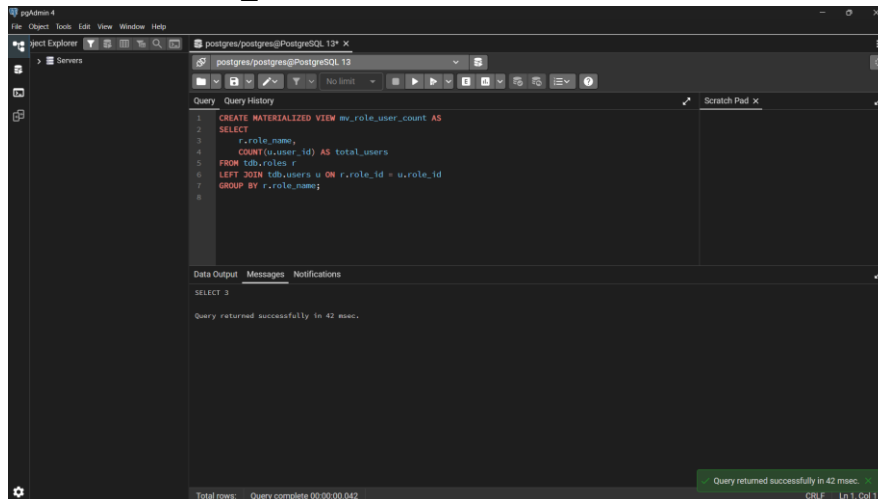


2. CREATE VIEW vw_recent_active_users AS
SELECT *
FROM tdb.users
WHERE is_active = true
AND created_date >= CURRENT_DATE - 30;

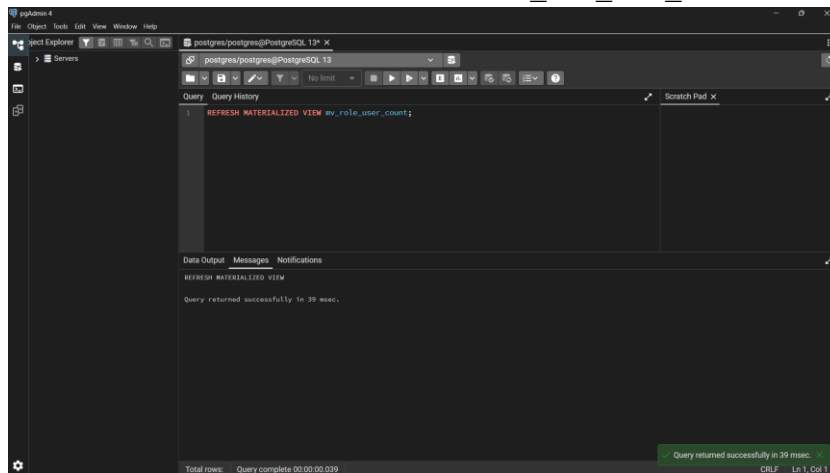


Work done by: Mitesh Bhagat

3. CREATE MATERIALIZED VIEW mv_role_user_count AS
SELECT
r.role_name,
COUNT(u.user_id) AS total_users
FROM tdb.roles r
LEFT JOIN tdb.users u ON r.role_id = u.role_id
GROUP BY r.role_name;

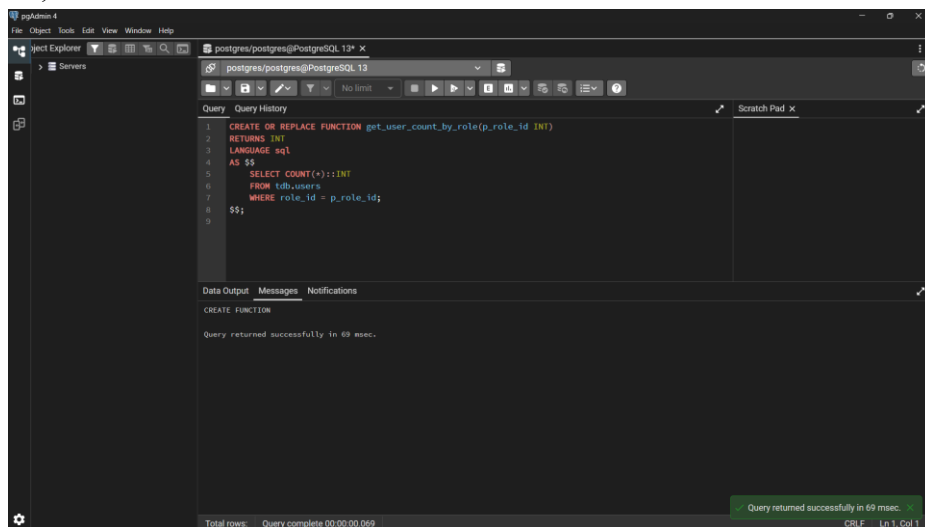


4. REFRESH MATERIALIZED VIEW mv_role_user_count;



2. Functions

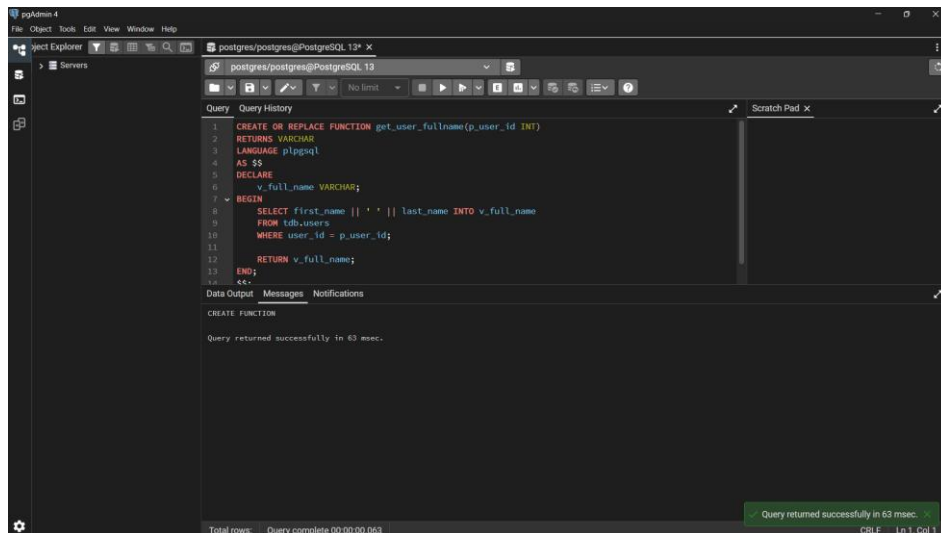
1. CREATE OR REPLACE FUNCTION get_user_count_by_role(p_role_id INT)
RETURNS INT
LANGUAGE sql
AS \$\$
SELECT COUNT(*)::INT
FROM tdb.users
WHERE role_id = p_role_id;
\$\$;



2. CREATE OR REPLACE FUNCTION get_user_fullname(p_user_id INT)
RETURNS VARCHAR
LANGUAGE plpgsql
AS \$\$
DECLARE
 v_full_name VARCHAR;
BEGIN
 SELECT first_name || ' ' || last_name INTO v_full_name
 FROM tdb.users
 WHERE user_id = p_user_id;

 RETURN v_full_name;
END;
\$\$;

Work done by: Mitesh Bhagat

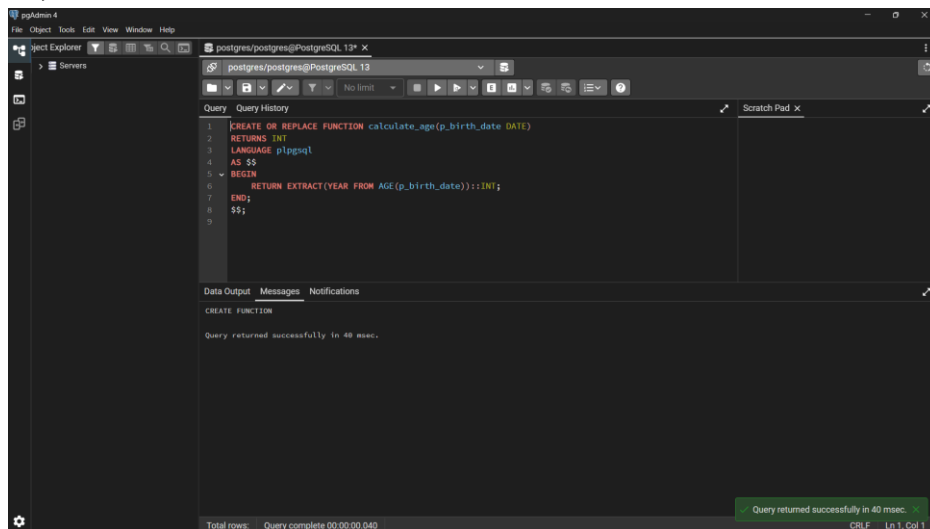


The screenshot shows the pgAdmin 4 interface with a query window open. The query is a PL/pgSQL function named `get_user_fullname` that takes a user ID as input and returns the full name. The query has been executed successfully, as indicated by the message "Query returned successfully in 63 msec." in the Data Output pane.

```
1 CREATE OR REPLACE FUNCTION get_user_fullname(p_user_id INT)
2 RETURNS VARCHAR
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6     v_full_name VARCHAR;
7 BEGIN
8     SELECT first_name || ' ' || last_name INTO v_full_name
9     FROM tdb.users
10    WHERE user_id = p_user_id;
11     RETURN v_full_name;
12 END;
13
```

Query returned successfully in 63 msec.

3. CREATE OR REPLACE FUNCTION calculate_age(p_birth_date DATE)
RETURNS INT
LANGUAGE plpgsql
AS \$\$
BEGIN
 RETURN EXTRACT(YEAR FROM AGE(p_birth_date))::INT;
END;
\$;



The screenshot shows the pgAdmin 4 interface with a query window open. The query is a PL/pgSQL function named `calculate_age` that takes a birth date as input and returns the age in years. The query has been executed successfully, as indicated by the message "Query returned successfully in 48 msec." in the Data Output pane.

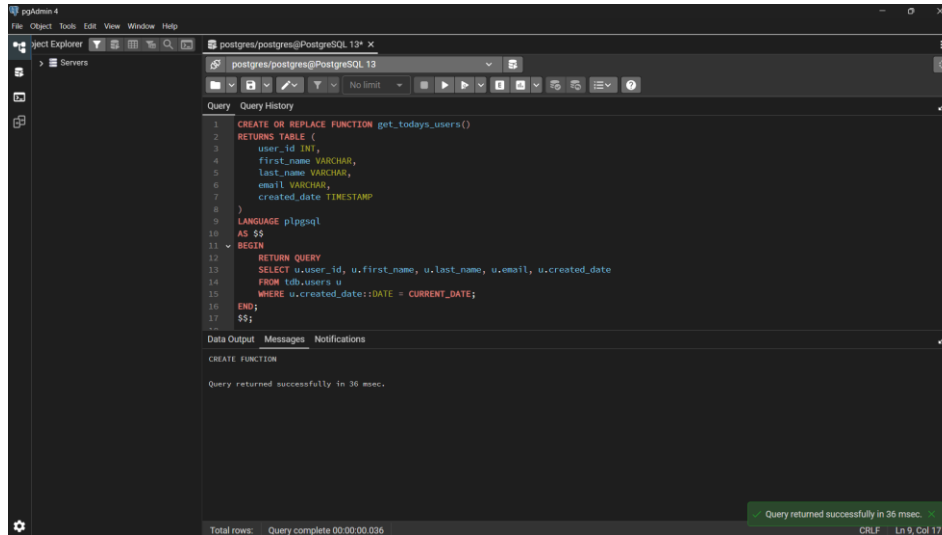
```
1 CREATE OR REPLACE FUNCTION calculate_age(p_birth_date DATE)
2 RETURNS INT
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     RETURN EXTRACT(YEAR FROM AGE(p_birth_date))::INT;
7 END;
8 $;
```

Query returned successfully in 48 msec.

4. CREATE OR REPLACE FUNCTION get_todays_users()
RETURNS TABLE (
 user_id INT,
 first_name VARCHAR,
 last_name VARCHAR,
 email VARCHAR,
 created_date TIMESTAMP
)
LANGUAGE plpgsql
AS \$\$

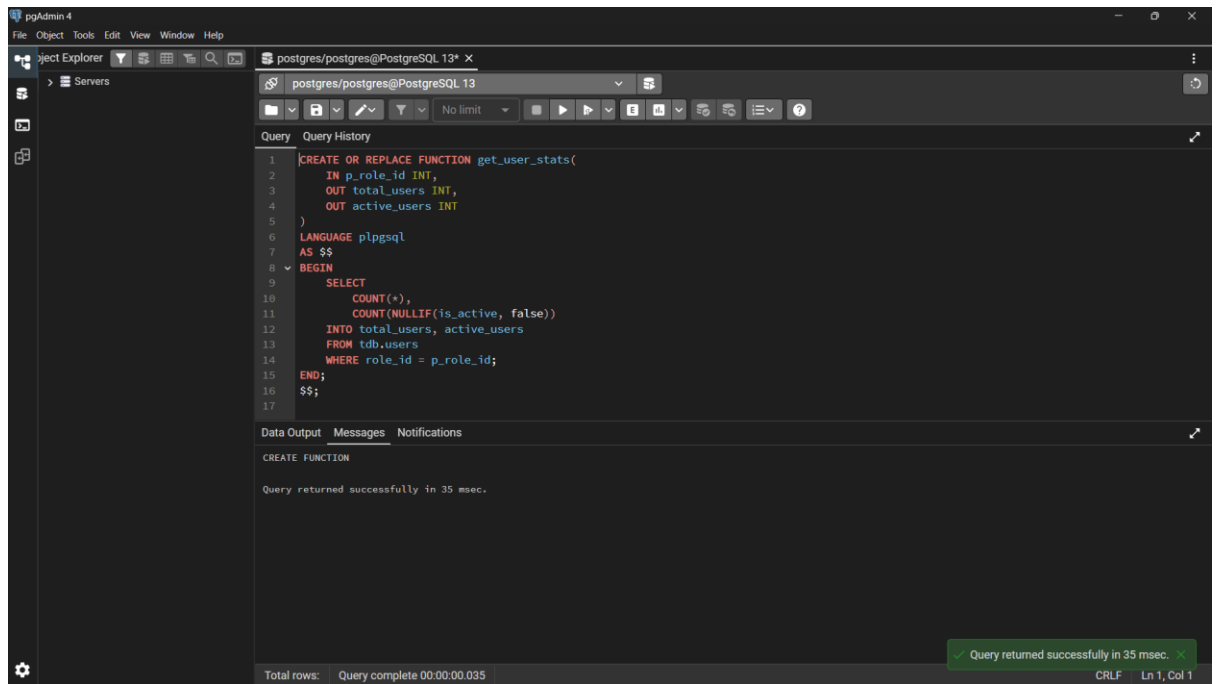
Work done by: Mitesh Bhagat

```
BEGIN
RETURN QUERY
SELECT u.user_id, u.first_name, u.last_name, u.email, u.created_date
FROM tdb.users u
WHERE u.created_date::DATE = CURRENT_DATE;
END;
$$;
```



5. CREATE OR REPLACE FUNCTION get_user_stats(
 IN p_role_id INT,
 OUT total_users INT,
 OUT active_users INT
)
LANGUAGE plpgsql
AS \$\$
BEGIN
SELECT
 COUNT(*),
 COUNT(NULLIF(is_active, false))
INTO total_users, active_users
FROM tdb.users
WHERE role_id = p_role_id;
END;
\$\$;

Work done by: Mitesh Bhagat



The screenshot displays the pgAdmin 4 web interface. On the left, the 'Object Explorer' pane shows a tree structure with 'Servers' expanded. The main pane is titled 'postgres/postgres@PostgreSQL 13' and contains a SQL query editor. The query is a PL/pgSQL function definition for 'get_user_stats'. Below the query editor, the 'Data Output' tab is active, showing the message 'CREATE FUNCTION' and 'Query returned successfully in 35 msec.'. A status bar at the bottom indicates 'Total rows: Query complete 00:00:00.035' and 'CRLF Ln 1, Col 1'.

```
1 CREATE OR REPLACE FUNCTION get_user_stats(  
2     IN p_role_id INT,  
3     OUT total_users INT,  
4     OUT active_users INT  
5 )  
6 LANGUAGE plpgsql  
7 AS $$  
8 BEGIN  
9     SELECT  
10        COUNT(*),  
11        COUNT(NULLIF(is_active, false))  
12    INTO total_users, active_users  
13    FROM tdb.users  
14    WHERE role_id = p_role_id;  
15 END;  
16 $$;  
17
```

CREATE FUNCTION
Query returned successfully in 35 msec.

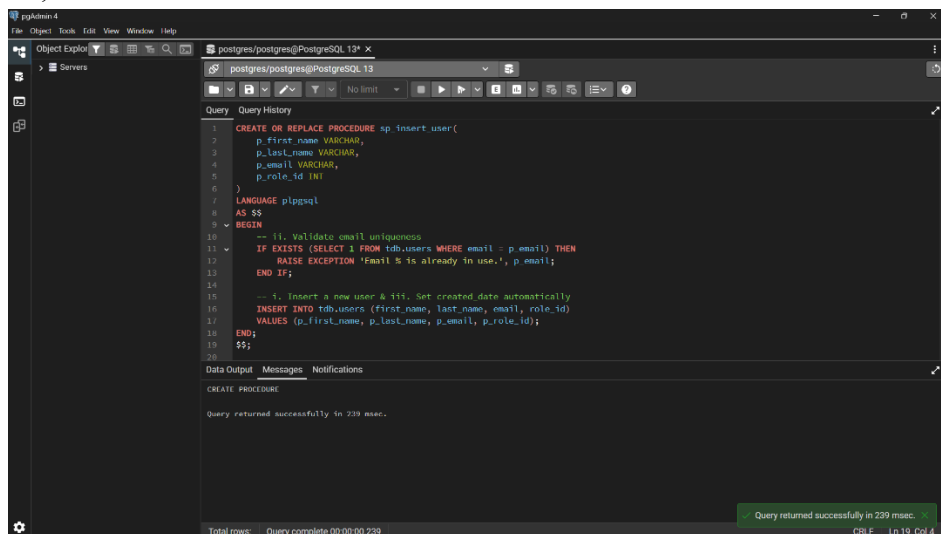
Total rows: Query complete 00:00:00.035 CRLF Ln 1, Col 1

Work done by: Mitesh Bhagat

3. Stored Procedures

a. Create a stored procedure to:

```
CREATE OR REPLACE PROCEDURE sp_insert_user(  
    p_first_name VARCHAR,  
    p_last_name VARCHAR,  
    p_email VARCHAR,  
    p_role_id INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- ii. Validate email uniqueness  
    IF EXISTS (SELECT 1 FROM tdb.users WHERE email = p_email) THEN  
        RAISE EXCEPTION 'Email % is already in use.', p_email;  
    END IF;  
  
    -- i. Insert a new user & iii. Set created_date automatically  
    INSERT INTO tdb.users (first_name, last_name, email, role_id)  
    VALUES (p_first_name, p_last_name, p_email, p_role_id);  
END;  
$;
```



Work done by: Mitesh Bhagat

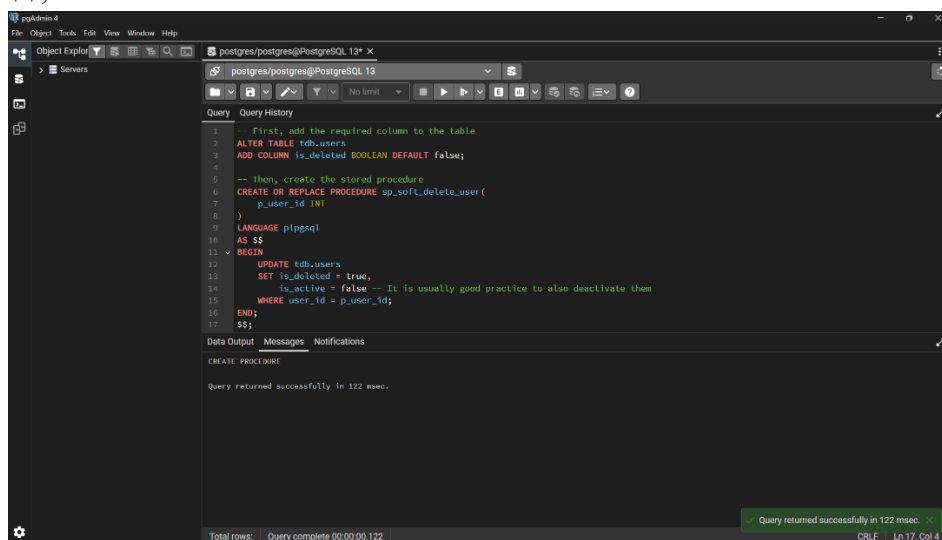
b. Create a stored procedure that: i. Soft deletes a user (is_deleted = true)

-- First, add the required column to the table

```
ALTER TABLE tdb.users  
ADD COLUMN is_deleted BOOLEAN DEFAULT false;
```

-- Then, create the stored procedure

```
CREATE OR REPLACE PROCEDURE sp_soft_delete_user(  
    p_user_id INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE tdb.users  
    SET is_deleted = true,  
        is_active = false -- It is usually good practice to also deactivate them  
    WHERE user_id = p_user_id;  
END;  
$$;
```



c. Create a stored procedure that UPDATE user role and Logs old role and new role into an audit table

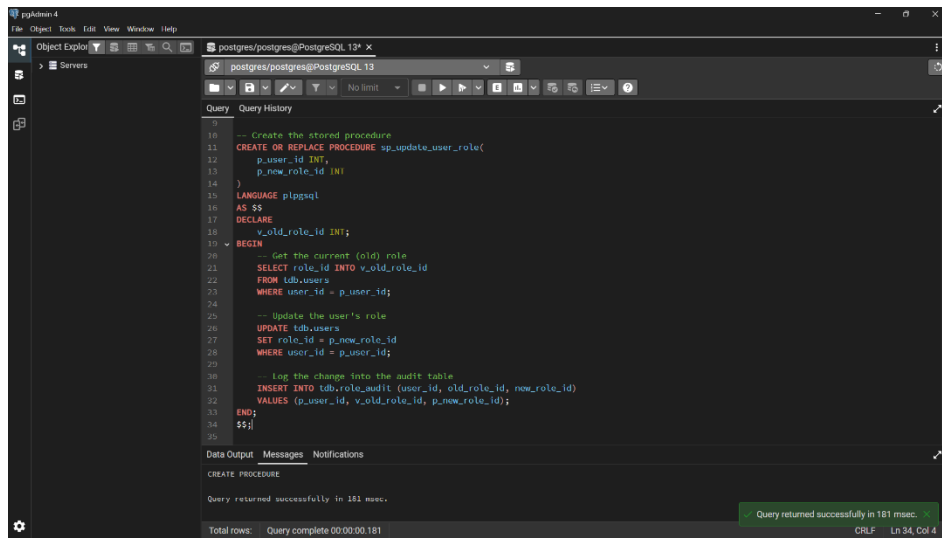
-- Create the audit table

```
CREATE TABLE tdb.role_audit (  
    audit_id SERIAL PRIMARY KEY,  
    user_id INT,  
    old_role_id INT,  
    new_role_id INT,  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Create the stored procedure

```
CREATE OR REPLACE PROCEDURE sp_update_user_role(  
    p_user_id INT,  
    p_new_role_id INT  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_old_role_id INT;  
BEGIN  
    -- Get the current (old) role  
    SELECT role_id INTO v_old_role_id  
    FROM tdb.users  
    WHERE user_id = p_user_id;  
  
    -- Update the user's role  
    UPDATE tdb.users  
    SET role_id = p_new_role_id  
    WHERE user_id = p_user_id;  
  
    -- Log the change into the audit table  
    INSERT INTO tdb.role_audit (user_id, old_role_id, new_role_id)  
    VALUES (p_user_id, v_old_role_id, p_new_role_id);  
END;  
$$;
```

Work done by: Mitesh Bhagat



The screenshot shows the pgAdmin 4 web interface. The main pane displays a SQL query for creating a stored procedure named `sp_update_user_role`. The query is as follows:

```
10 -- Create the stored procedure
11 CREATE OR REPLACE PROCEDURE sp_update_user_role(
12     p_user_id INT,
13     p_new_role_id INT
14 )
15 LANGUAGE plpgsql
16 AS $$
17 DECLARE
18     v_old_role_id INT;
19 BEGIN
20     -- Get the current (old) role
21     SELECT role_id INTO v_old_role_id
22     FROM tdb_users
23     WHERE user_id = p_user_id;
24
25     -- Update the user's role
26     UPDATE tdb_users
27     SET role_id = p_new_role_id
28     WHERE user_id = p_user_id;
29
30     -- Log the change into the audit table
31     INSERT INTO tdb_role_audit (user_id, old_role_id, new_role_id)
32     VALUES (p_user_id, v_old_role_id, p_new_role_id);
33 END;
34 $$;
```

The bottom pane shows the query execution results, indicating that the procedure was created successfully in 181 msec.

Query returned successfully in 181 msec.

Total rows: Query complete 00:00:00.181

4. Stored Procedures

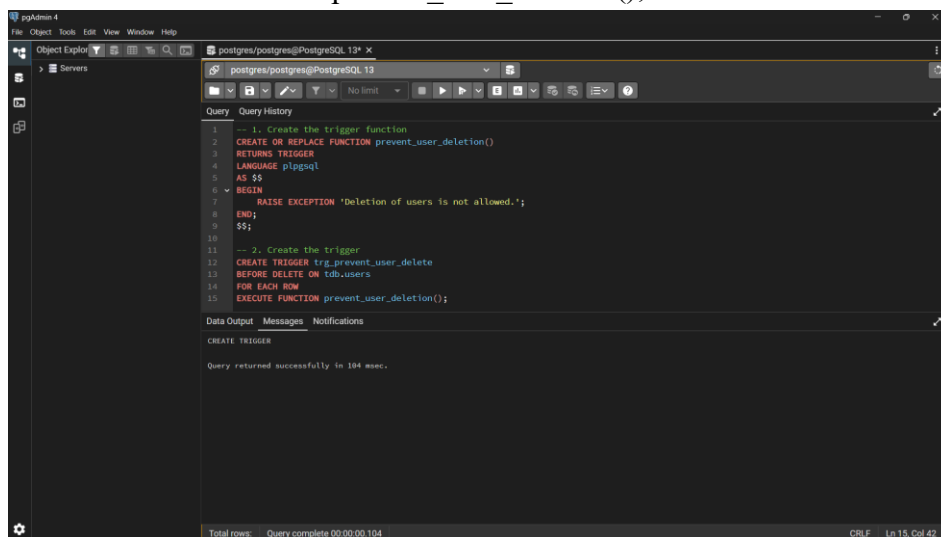
1. Create a trigger that Automatically updates modified_date on UPDATE of users table

-- 1. Create the trigger function

```
CREATE OR REPLACE FUNCTION prevent_user_deletion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION 'Deletion of users is not allowed.';
END;
$$;
```

-- 2. Create the trigger

```
CREATE TRIGGER trg_prevent_user_delete
BEFORE DELETE ON tdb.users
FOR EACH ROW
EXECUTE FUNCTION prevent_user_deletion();
```



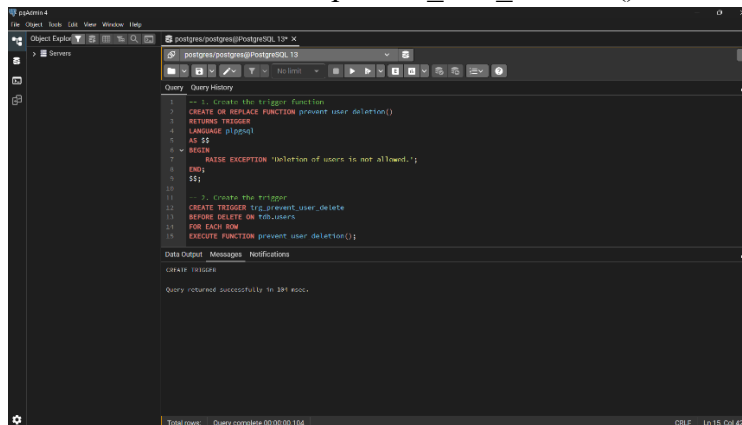
2. Create a trigger that: i. Prevents deletion of users ii. Raises an exception if DELETE is attempted

-- 1. Create the trigger function

```
CREATE OR REPLACE FUNCTION prevent_user_deletion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION 'Deletion of users is not allowed.';
END;
$$;
```

-- 2. Create the trigger

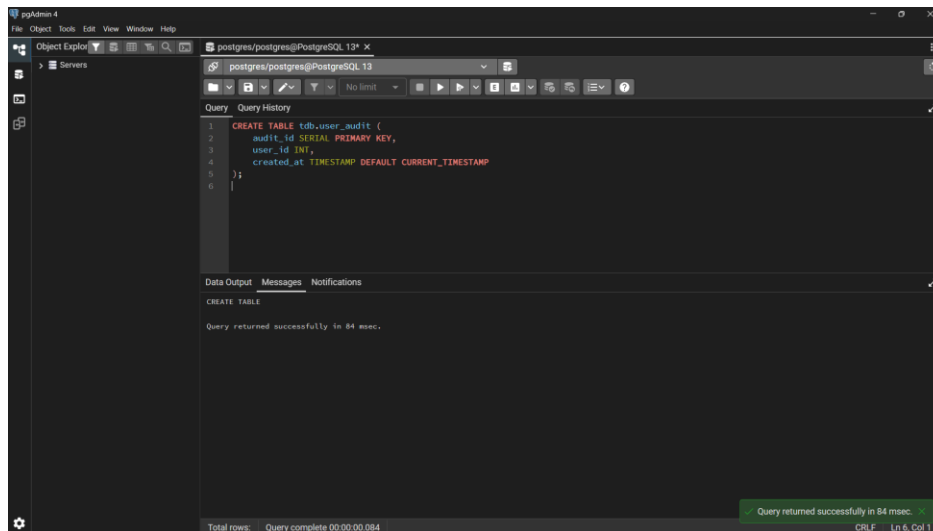
```
CREATE TRIGGER trg_prevent_user_delete
BEFORE DELETE ON tdb.users
FOR EACH ROW
EXECUTE FUNCTION prevent_user_deletion();
```



3. Create an AFTER INSERT trigger to: i. Log user creation into an audit table

```
CREATE TABLE tdb.user_audit (
    audit_id SERIAL PRIMARY KEY,
    user_id INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Work done by: Mitesh Bhagat



-- 1. Create the trigger function

CREATE OR REPLACE FUNCTION log_user_creation()

RETURNS TRIGGER

LANGUAGE plpgsql

AS \$\$

BEGIN

INSERT INTO tdb.user_audit (user_id)

VALUES (NEW.user_id);

RETURN NEW;

END;

\$\$;

-- 2. Create the AFTER INSERT trigger

CREATE TRIGGER trg_log_user_creation

AFTER INSERT ON tdb.users

FOR EACH ROW

EXECUTE FUNCTION log_user_creation();

Work done by: Mitesh Bhagat

```
-- 1. Create the trigger function
CREATE OR REPLACE FUNCTION log_user_creation()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO tdb.user_audit (user_id)
    VALUES (NEW.user_id);
    RETURN NEW;
END;
$$;

-- 2. Create the AFTER INSERT trigger
CREATE TRIGGER trg_log_user_creation
AFTER INSERT ON tdb.users
FOR EACH ROW
EXECUTE FUNCTION log_user_creation();
```

CREATE TRIGGER

Query returned successfully in 72 msec.

Total rows: Query complete 00:00:00.072

5. Cursors

DO \$\$

DECLARE

-- Define the cursor to fetch specific columns

cur_users CURSOR FOR SELECT user_id, email FROM tdb.users;

rec_user RECORD;

BEGIN

-- Loop through the cursor results

FOR rec_user IN cur_users LOOP

-- Print the values using RAISE NOTICE

RAISE NOTICE 'User ID: %, Email: %', rec_user.user_id, rec_user.email;

END LOOP;

END;

\$\$;

```
DO $$
DECLARE
    -- Define the cursor to fetch specific columns
    cur_users CURSOR FOR SELECT user_id, email FROM tdb.users;
    rec_user RECORD;
BEGIN
    -- Loop through the cursor results
    FOR rec_user IN cur_users LOOP
        -- Print the values using RAISE NOTICE
        RAISE NOTICE 'User ID: %, Email: %', rec_user.user_id, rec_user.email;
    END LOOP;
END;
$$;
```

NOTICE: User ID: 1, Email: john@example.com
NOTICE: User ID: 2, Email: jane@example.com
NOTICE: User ID: 3, Email: bob@example.com
DO

Query returned successfully in 132 msec.

Total rows: Query complete 00:00:00.132

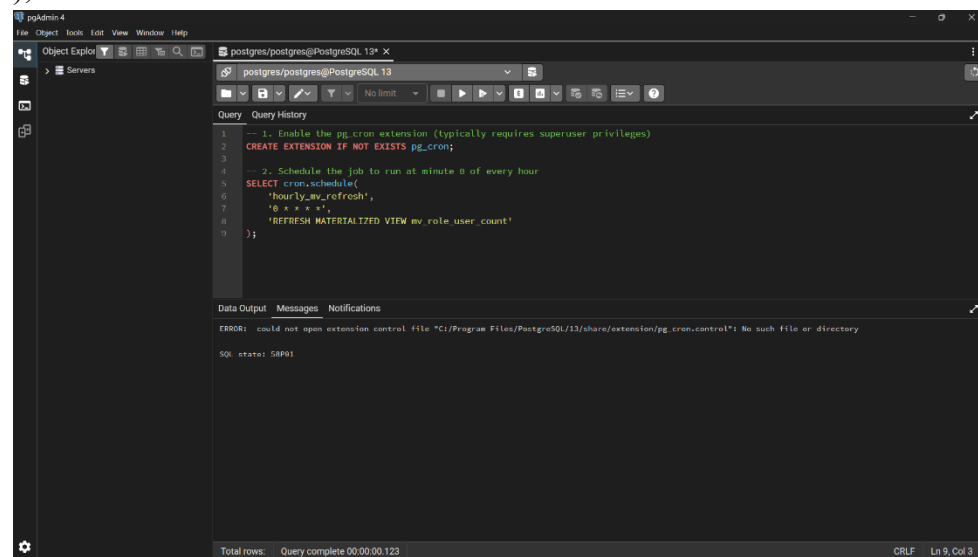
Work done by: Mitesh Bhagat

6. Jobs / Scheduling

-- 1. Enable the pg_cron extension (typically requires superuser privileges)
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- 2. Schedule the job to run at minute 0 of every hour

```
SELECT cron.schedule(  
    'hourly_mv_refresh',  
    '0 * * * *',  
    'REFRESH MATERIALIZED VIEW mv_role_user_count'  
);
```



Assignment 4 - PostgreSQL Basics

1. Transactions & ACID

a) BEGIN;

-- I. Insert a new user

```
INSERT INTO tdb.users (first_name, last_name, email, role_id)
VALUES ('Test', 'User', 'testuser@example.com', 1);
```

-- II. Update user status

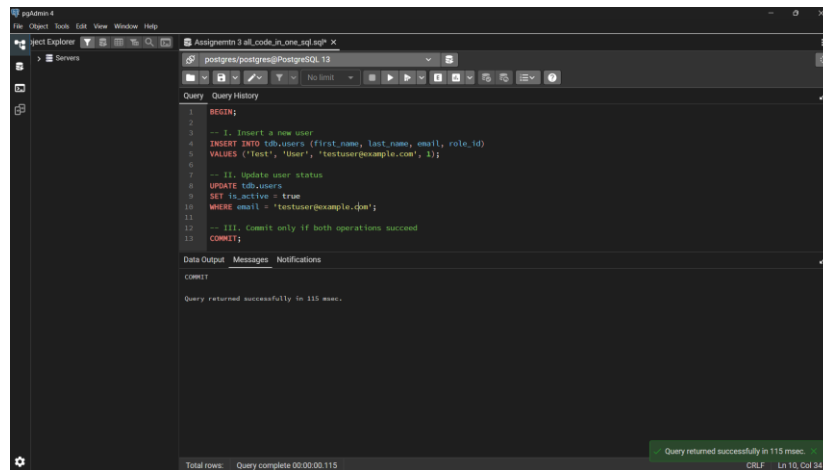
```
UPDATE tdb.users
```

```
SET is_active = true
```

```
WHERE email = 'testuser@example.com';
```

-- III. Commit only if both operations succeed

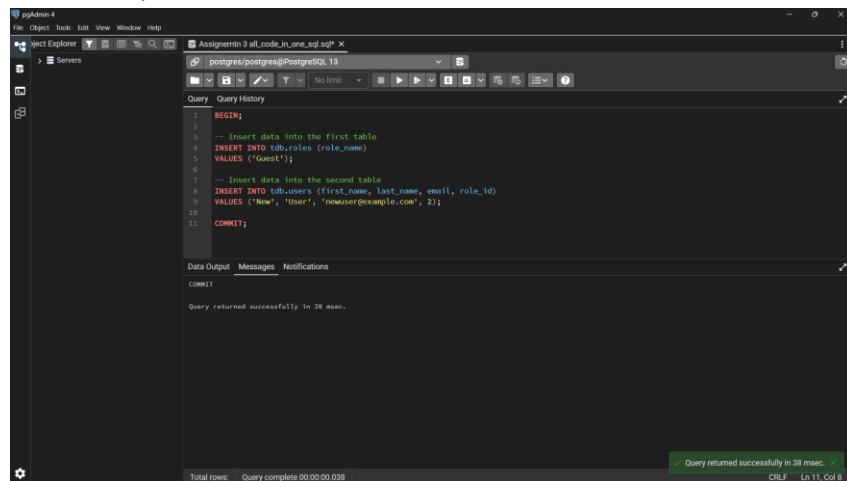
```
COMMIT;
```



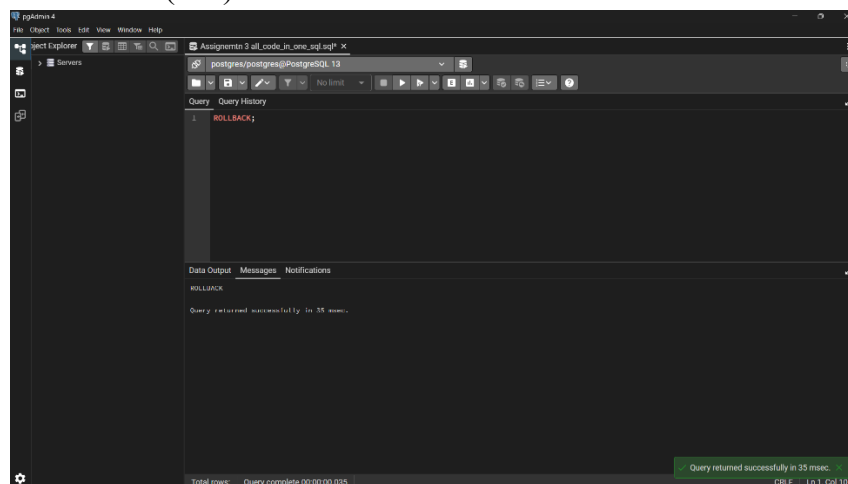
Work done by: Mitesh Bhagat

b) There was some Issue so I did “ROLLBACK” then I write this code:

```
BEGIN;  
-- Insert data into the first table  
INSERT INTO tdb.roles (role_name)  
VALUES ('Guest');  
-- Insert data into the second table  
INSERT INTO tdb.users (first_name, last_name, email, role_id)  
VALUES ('New', 'User', 'newuser@example.com', 2);  
COMMIT;
```

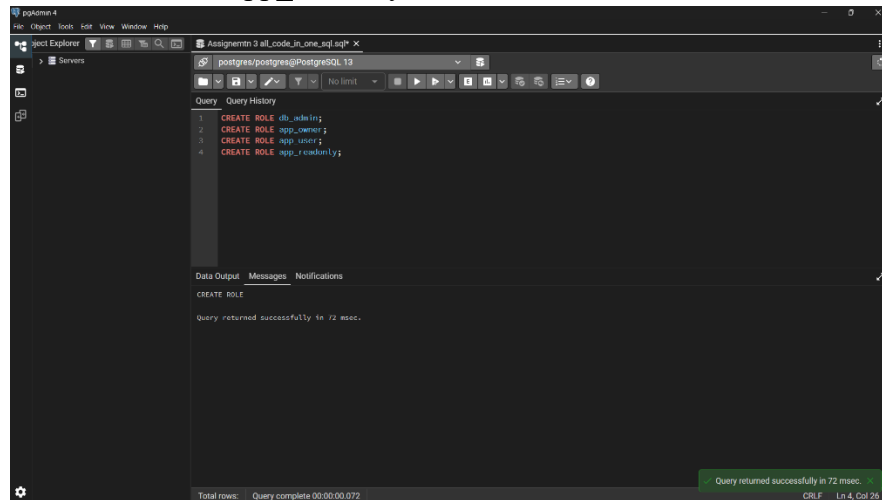


c) Rolls back if any statement fails
=> Done in (2-B)

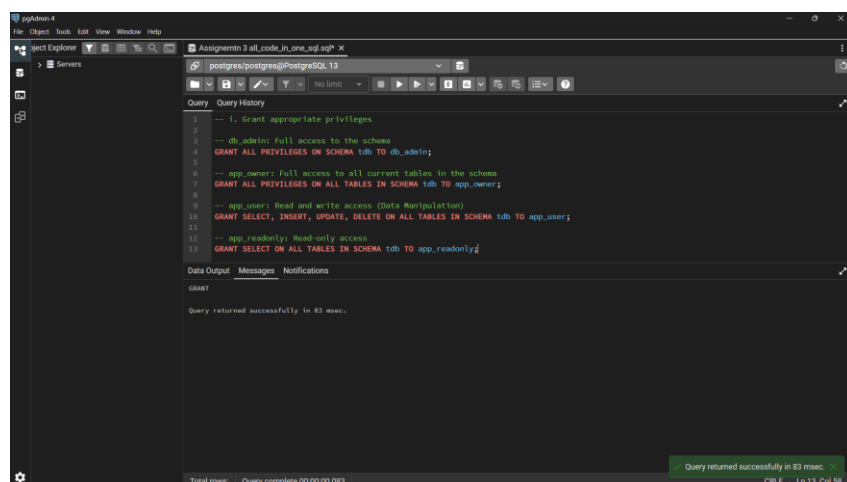


2. Security Basics

- a) CREATE ROLE db_admin;
CREATE ROLE app_owner;
CREATE ROLE app_user;
CREATE ROLE app_readonly;



- b) -- i. Grant appropriate privileges
-- db_admin: Full access to the schema
GRANT ALL PRIVILEGES ON SCHEMA tdb TO db_admin;
-- app_owner: Full access to all current tables in the schema
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA tdb TO app_owner;
-- app_user: Read and write access (Data Manipulation)
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA tdb TO app_user;
-- app_readonly: Read-only access
GRANT SELECT ON ALL TABLES IN SCHEMA tdb TO app_readonly;



Work done by: Mitesh Bhagat

c) -- i. Revoke privileges

-- db_admin

```
REVOKE ALL PRIVILEGES ON SCHEMA tdb FROM db_admin;
```

-- app_owner

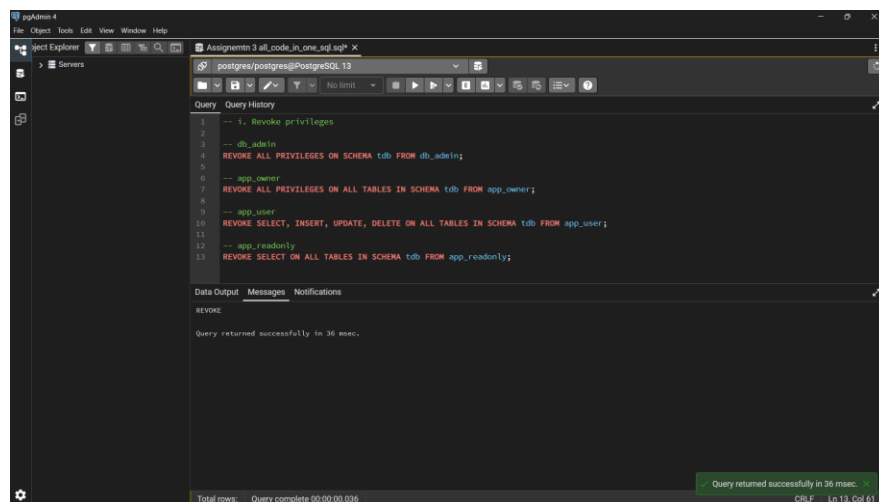
```
REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA tdb FROM  
app_owner;
```

-- app_user

```
REVOKE SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN  
SCHEMA tdb FROM app_user;
```

-- app_readonly

```
REVOKE SELECT ON ALL TABLES IN SCHEMA tdb FROM  
app_readonly;
```



Work done by: Mitesh Bhagat

d) -- i. Schema-level security

-- Controls access to the schema object itself

GRANT USAGE, CREATE ON SCHEMA tdb TO db_admin;

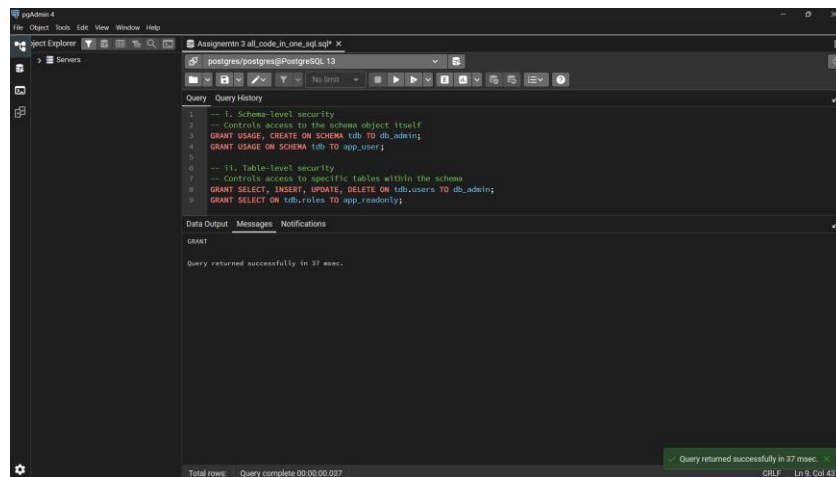
GRANT USAGE ON SCHEMA tdb TO app_user;

-- ii. Table-level security

-- Controls access to specific tables within the schema

GRANT SELECT, INSERT, UPDATE, DELETE ON tdb.users TO db_admin;

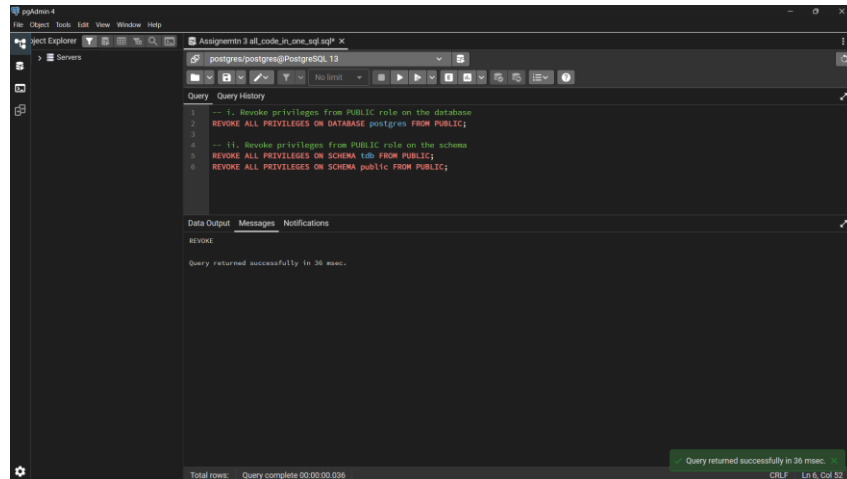
GRANT SELECT ON tdb.roles TO app_readonly;



Work done by: Mitesh Bhagat

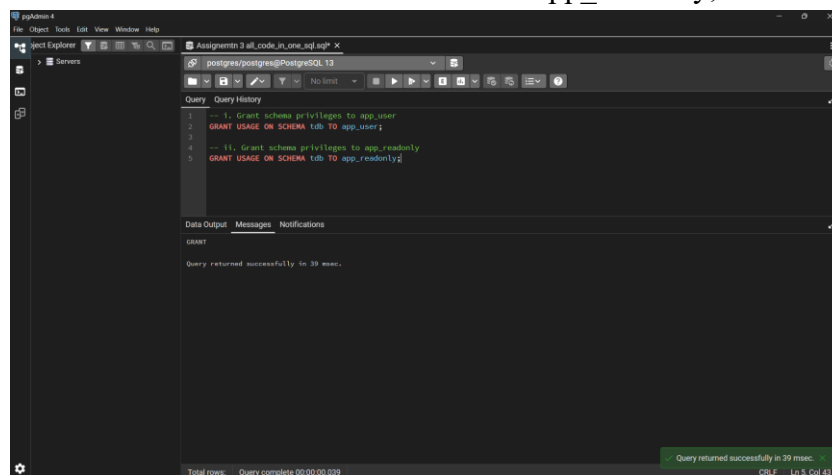
- e) -- i. Revoke privileges from PUBLIC role on the database
REVOKE ALL PRIVILEGES ON DATABASE postgres FROM PUBLIC;

-- ii. Revoke privileges from PUBLIC role on the schema
REVOKE ALL PRIVILEGES ON SCHEMA tdb FROM PUBLIC;
REVOKE ALL PRIVILEGES ON SCHEMA public FROM PUBLIC;



- f) -- i. Grant schema privileges to app_user
GRANT USAGE ON SCHEMA tdb TO app_user;

-- ii. Grant schema privileges to app_readonly
GRANT USAGE ON SCHEMA tdb TO app_readonly;



g) Table & Object Privileges

-- i. Grant permissions to app_user

```
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES  
IN SCHEMA hc TO app_user;
```

-- ii. Grant read-only permissions to app_readonly

```
GRANT SELECT ON ALL TABLES IN SCHEMA hc TO  
app_readonly;
```

-- iii. Prevent app_user from dropping or truncating tables

-- Note: A user cannot DROP a table unless they are the table owner or a superuser.

-- We explicitly revoke TRUNCATE to ensure that specific privilege is removed.

```
REVOKE TRUNCATE ON ALL TABLES IN SCHEMA hc FROM  
app_user;
```

